



COMSATS University Islamabad, Pakistan

OMNIRAID

**A project presented to
COMSATS Institute of Information Technology, Islamabad
Sahiwal Campus**

**In partial fulfillment
of the requirement for the degree of**

Bachelor of Science in Software Engineering (2022-2026)

By

Muhammad Sameed	CIIT/FA22-BSE-061/SWL
Zohaib Iqbal	CIIT/FA22-BSE-053/SWL

DECLARATION

We hereby declare that this software, neither whole nor as a part has been copied out from any source. It is further declared that we have developed this software and accompanied report entirely on the basis of our personal efforts. If any part of this project is proved to be copied out from any source or found to be reproduction of some other. We will stand by the consequences. No Portion of the work presented has been submitted of any application for any other degree or qualification of this or any other university or institute of learning.

Muhammad Sameed

FA22-BSE-061

Zohaib Iqbal

FA22-BSE-053

CERTIFICATE OF APPROVAL

It is to certify that the final year project of BS (SE) “OMNIRAID” was developed by **Muhammad Sameed (CIIT/FA22-BSE -061/SWL)** and **Zohaib Iqbal (CIIT/FA22-BSE-053/SWL)** under the supervision of “Mr. Ali Sher Kashif” and co- supervisor “Mr. Ali Sher Kashif” and that in his opinion; it is fully adequate, in scope and quality for the degree of Bachelor of Science in Software Engineering.

Supervisor

Sir Ali Sher Kashif

Lecturer

Department of Computer Science

COMSATS University Islamabad, Sahiwal Campus

External Examiner

Dr Javeed Rashid

Deputy Registrar IT

University of Okara

Head of Department

Dr. Zafar Iqbal Roy

Assistant Professor

Department of Computer Science

COMSATS University Islamabad, Sahiwal Campus

Executive Summary

In modern gaming environments, multiplayer first-person shooter (FPS) games have gained significant popularity among players worldwide. However, many existing FPS games face critical issues such as lag, poor matchmaking, unbalanced gameplay, and lack of fair competition. These limitations negatively impact the overall gaming experience and reduce player engagement. Additionally, maintaining smooth real-time performance and ensuring secure gameplay remains a major challenge in current systems.

Current multiplayer systems often rely on inefficient networking models and lack proper optimization, which leads to latency issues and unstable connections. Moreover, many games do not provide balanced matchmaking systems, resulting in unfair competition between players of different skill levels. The absence of effective anti-cheat mechanisms further affects the integrity of gameplay.

To address these challenges, OMNIRAID is proposed as a PC-based online multiplayer FPS game developed using modern game development technologies. The system is designed to provide a fast-paced, smooth, and competitive gaming experience by incorporating real-time multiplayer functionality, fair matchmaking, and optimized performance. It also focuses on ensuring secure gameplay through anti-cheat measures and stable server infrastructure.

The proposed system utilizes Photon Fusion [1] for real-time networking and PlayFab for backend management, including player data, leaderboards, and progression systems. The game includes multiple features such as AI bots for filling matches, various game modes including Team Deathmatch, Free-for-All, and Capture the Flag, and advanced combat mechanics with realistic shooting behavior. Additionally, players are provided with customization options, ranking systems, and interactive maps designed using Blender.

OMNIRAID is implemented using Unity as the core game engine, with C# as the primary programming language. Supporting tools such as Blender, Adobe Photoshop, and After Effects are used for 3D modeling, design, and animations. The system is designed as a standalone PC game that allows users to engage in real-time multiplayer matches with a seamless and immersive experience.

The expected outcome of this project is a well-optimized, responsive, and engaging multiplayer FPS game that overcomes common issues in existing systems. OMNIRAID aims to deliver a fair, competitive, and enjoyable gaming platform while also providing valuable practical experience in game development, networking, and system design.

Acknowledgement

All praise is to Almighty Allah who bestowed upon us a minute portion of His boundless knowledge by virtue of which we were able to accomplish this challenging task.

We are greatly indebted to our project supervisor “Mr. Ali Sher Kashif” and our Co- Supervisor “Mr. Ali Sher Kashif”. Without their personal supervision, advice and valuable guidance, completion of this project would have been doubtful. We are deeply indebted to them for their encouragement and continual help during this work.

And we are also thankful to our parents and family who have been a constant source of encouragement for us and brought us the values of honesty & hard work.

Muhammad Sameed

FA22-BSE-061

Zohaib Iqbal

FA22-BSE-053

Abbreviations

SRS	Software Requirement Specifications
SDD	Software Design Description
C#	C-Sharp Programming Language
CTF	Capture the Flag
CSGO2	Counter-Strike: Global Offensive 2
ECS	Entity Component System
FFA	Free-for-All
FPS	First Person Shooter
FYP	Final Year Project
HUD	Heads Up Display
MMR	Matchmaking Rating
MVC	Model-View-Controller
NavMesh	Navigation Mesh
OOP	Object-Oriented Programming
TDM	Team Death Match
UML	Unified Modeling Language
XP	Experience Points
GPU	Graphics Processing Unit

Table of Contents

1.	Introduction.....	1
1.1.	Brief.....	2
1.1.1.	Purpose of Project.....	3
1.1.2.	Scope of the System.....	3
1.1.3.	Technologies Used.....	3
1.1.4.	System Overview:.....	3
1.1.5.	Outcome of the Project	4
1.2.	Relevance to Course Module	4
1.2.1.	Software Engineering	4
1.2.2.	Database System	5
1.2.3.	Game Development	5
1.2.4.	Object Oriented Programming	5
1.2.5.	Data Structure and Algorithms.....	6
1.2.6.	Human Computer Interaction.....	6
1.2.7.	Information Security	6
1.2.8.	Computer Networks	6
1.2.9.	Final Year Project I and II	6
1.3.	Project Background.....	7
1.4.	Literature Review	8
1.5.	Analysis from Literature Review.....	10
1.5.1.	Alignment with Modern Multiplayer Gaming Trends	10
1.5.2.	Real-Time Networking and Synchronization Analysis	10
1.5.3.	Comparison with Existing Matchmaking Systems.....	10
1.5.4.	Backend Data Management and Player Progression	11
1.5.5.	Application of Security in Multiplayer Systems.....	11
1.5.6.	User Experience in FPS Gameplay	11

1.5.7 Addressing Limitations of Existing Multiplayer FPS Systems.....	11
1.6. Methodology and Software Lifecycle	11
1.6.1. Methodology.....	12
1.6.2. Agile Practices Applied	12
1.6.3. Software Development Life Cycle (SDLC) Model.....	12
1.6.4. Incremental Delivery	13
1.6.5. Development Tools and Environment	13
1.6.6. Testing and Quality Assurance	13
1.6.7. Advantages of the Selected Methodology and SDLC.....	13
2. Problem Definition	15
2.1. Problem Statement.....	17
2.2. Deliverable and Development Requirements.....	17
2.2.1 Deliverables.....	17
2.2.2. Development Requirements:.....	19
2.3. Current System	20
2.3.1. Counter-Strike: Global Offensive (CSGO2)	20
2.3.2. Valorant	21
2.3.3. Call of Duty: Warzone	22
2.3.4. Comparative Analysis of Existing Systems	23
2.3.5. Identified Limitations	23
2.3.6. Gap Analysis and Motivation for OmniRaid.....	23
3. Requirement Analysis	26
3.1. Introduction to Requirement Analysis	26
3.2. System Overview	26
3.3. Actors in the System	27
3.3.1. Player	27
3.3.2. AI Bot.....	27
3.3.3. System / Server	27
3.4. Usecase Diagrams	27

3.5.	Use Case Descriptions	31
3.5.1.	AI System	31
3.5.2.	Gameplay	32
3.5.3.	Multiplayer System	33
3.5.5.	Progression System	35
3.6.	Functional Requirements	35
3.6.1.	Menu System Requirements	35
3.6.2.	Gameplay Requirements	35
3.6.3.	Multiplayer Requirements	36
3.6.4.	AI Bot Requirements	36
3.6.5.	Progression System Requirements	37
3.7.	Non-Functional Requirements	37
3.7.1.	Performance Requirements	37
3.7.2.	Usability Requirements	37
3.7.3.	Reliability Requirements	37
3.7.4.	Security Requirements	38
3.7.5.	Compatibility Requirements	38
3.7.6.	Scalability Requirements	38
3.7.7.	Maintainability	38
4.	Design And Architecture	40
4.1.	System Architecture	40
4.1.1.	Architecture Style	40
4.1.2.	Components	40
4.1.3.	Interactions	41
4.1.4.	System Architecture Diagram	42
4.2.	Data Representation	42
4.2.1.	Data storage approach	43
4.2.2.	Data Dictionary	44
4.3.	Process Flow	46

4.3.1.	Gameplay Flow:	46
4.3.2.	Shooting Flow:.....	48
4.4.	Design Models	49
4.4.1.	Class Diagram.....	49
4.4.2.	Sequence Diagrams.....	50
5.	Implementation	53
5.1.	Algorithms	53
5.1.1.	AI Pathfinding Algorithm	53
5.1.2.	AI Decision Algorithm	53
5.1.3.	Hit Registration & Lag Compensation Algorithm	54
5.1.4.	Matchmaking Algorithm	54
5.1.5.	Progression (XP Calculation) Algorithm.....	55
5.2.	External API's:	56
5.3.	User Interface	57
5.3.1.	Main Menu	57
5.3.2.	Lobby	58
5.3.3.	Inventory	58
5.3.4.	Shop	59
5.3.5.	Containers	60
5.3.6.	Settings	61
5.3.7.	Match Result Screen	62
6.	Testing and Evaluation	65
6.1.	Manual Testing.....	65
6.1.1.	System Testing	65
6.1.2.	Unit Testing	66
6.1.3.	Functional Testing	68
6.1.4.	Integration Testing	70
6.2.	Automated Testing.....	72
7.	Conclusion	75

7.1.	Future Work.....	76
7.1.1.	Advanced Matchmaking System	76
7.1.2.	Dedicated Server Infrastructure.....	76
7.1.3.	Enhanced AI and Bot Intelligence	76
7.1.4.	Competitive Ranking and Leaderboards	76
7.1.5.	Cross-Platform Support	77
7.1.6.	Advanced Analytics and Player Insights	77
7.1.7.	In-Game Economy and Monetization.....	77
7.1.8.	Enhanced Security and Anti-Cheat Mechanisms	77
7.1.9.	Voice Chat and Social Features	77
7.1.10.	Dynamic Maps and Game Modes.....	77
8.	References.....	78

List of Figures

Figure 2.1: CS2 Lobby	20
Figure 2.2: CS2 Gameplay	21
Figure 2.3: Valorant Gameplay	21
Figure 2.4: Valorant Lobby	22
Figure 2.5: Call Of Duty Warzone Lobby	22
Figure 2.6: Call Of Duty Warzone Gameplay	23
Figure 3.7: OMNIRAID menu System	28
Figure 3.8: OMNIRAID GAMEPLAY	29
Figure 3.9: OMNIRAID AI Management System	29
Figure 3.10: OMNIRAID Multiplayer System	30
Figure 3.11: Player Progression System	30
Figure 4.1.4.12: OMNIRAID Architecture Diagram	42
Figure 4.2.1.1.13: PlayerProfile Data	43
Figure 4.2.1.2.14: LeaderBoard Data in PlayFab	43
Figure 4.2.1.3.15: MatchRecord Data	44
Figure 4.3.1.16: Activity Diagram of Gameplay System	47
Figure 4.3.2.17: Activity Diagram of Shooting System	48
Figure 4.4.1.18: Class Diagram of OmniRaid	49
Figure 4.4.2.19: Sequence Diagram of Omniraid	50
Figure 4.4.2.20: Sequence Diagram of Omniraid	51
Figure 5.3.1.21: Main-Menu	57
Figure 5.3.2.22: Looby Interface	58
Figure 5.3.3.23: Player Inventory	59
Figure 5.3.4.24: InGame Shop UI	60
Figure 5.3.5.25: Opening Interface	61
Figure 5.3.6.26: Setting Interface	62
Figure 5.3.7.27: Match Result Screen	63

List of Tables

Table 1.1: Gap Analysis and Proposed Solutions for Existing Multiplayer FPS Games in OmniRaid	9
Table 2.2: Comparison of Existing Multiplayer FPS Games with Identified Limitations	23
Table 3.3: UC-1.....	31
Table 3.4: UC-1.1	31
Table 3.5: UC-2.....	31
Table 3.6: UC-3.....	32
Table 3.7: UC-4.....	32
Table 3.8: UC-5.....	32
Table 3.9: UC-9.....	33
Table 3.10: UC-7	33
Table 3.11: UC-8	33
Table 3.12: UC-9	34
Table 3.13: UC-10	34
Table 3.14: UC-11	35
Table 3.6.1.15: FR-1	35
Table 3.6.2.16: FR-2	35
Table 3.6.3.17: FR-3	36
Table 3.6.4.18: FR-4	36
Table 3.6.5.19: FR-5	37
Table 5.2.20: Details of APIs Used in OmniRaid	56
Table 6.1.2.21: Unit Testing for OmniRaid System	67
Table 6.1.3.22: Functional Testing for OmniRaid System.....	69
Table 6.1.4.23: Integration Testing for OmniRaid System	71
Table 6.2.24: Tools Used for Automated Testing.....	72
Table 6.2.25: Automated Test Cases	73

Chapter 1

INTRODUCTION

1. Introduction

In recent years, the gaming industry has grown rapidly [2], and multiplayer first-person shooter (FPS) games have become one of the most popular forms of entertainment worldwide. Millions of players engage in these games daily for competitive and recreational purposes. However, despite their popularity, many existing FPS games still face several challenges that affect the overall player experience. Common issues include high latency (lag), unfair matchmaking, unbalanced gameplay, and inconsistent performance across different regions. These problems can reduce player satisfaction and make it difficult to maintain long-term engagement. In regions with limited or unstable internet connectivity, these issues become even more noticeable. Players often experience delays during gameplay, which directly affects reaction time and performance in competitive matches. Moreover, when matchmaking systems fail to properly balance teams based on skill level, it leads to unfair matches where some players dominate while others struggle. This imbalance discourages new players and reduces the overall quality of gameplay. Additionally, many modern FPS games require high-end hardware, making them less accessible for users with mid-range systems.

To address these challenges, **OmniRaid** is developed as an online multiplayer first-person shooter game that focuses on delivering a smooth, balanced, and enjoyable gaming experience. The game is designed for PC platforms and aims to provide fast-paced competitive gameplay with minimal latency and responsive controls. It includes multiple game modes such as Team Deathmatch, Free-for-All, and Capture the Flag, allowing players to choose different styles of gameplay based on their preferences. One of the key features of OmniRaid is its focus on fair and consistent gameplay. The system includes a matchmaking mechanism that aims to pair players with similar skill levels, ensuring balanced matches. In addition, AI-controlled bots are integrated into the system to fill empty player slots when necessary, so that matches remain active and competitive even when fewer players are available. This helps maintain continuity and improves the overall user experience.

From a technical perspective, OmniRaid is built using modern tools and technologies to ensure performance and scalability. The game is developed using Unity as the core game engine, which provides support for graphics, physics, and gameplay mechanics. Real-time multiplayer functionality is implemented using Photon Fusion, which helps manage networking, synchronization, and lag compensation. For backend services such as player data management, leaderboards, and progression tracking, PlayFab is used as a cloud-based solution. These technologies work together to create a reliable and efficient system for online gameplay. The game also provides features such as player progression, ranking systems, and customization options. Players can earn experience points, unlock new items, and personalize their loadouts according to their playstyle. The maps and environments are designed to be visually appealing while still optimized for performance, ensuring that the game can run smoothly on mid-range systems.

Overall, OmniRaid is designed to deliver a modern and refined multiplayer FPS experience by directly addressing the common limitations found in many existing games. Instead of focusing only on visuals or complex features, the system emphasizes core gameplay quality, including smooth performance, low latency, and fair competition. By optimizing networking and game mechanics [3], OmniRaid ensures that players experience responsive controls and real-time interactions, which are essential in fast-paced shooting environments. This focus on performance

is particularly important for players in regions where network conditions are not always stable, making the game more reliable and consistent under different conditions.

In addition to performance, the project places strong emphasis on balanced and fair gameplay. The implementation of skill-based matchmaking helps create competitive matches where players are paired with others of similar ability, reducing frustration and improving overall engagement. The inclusion of AI-controlled bots further strengthens the system by maintaining match continuity and ensuring that players are not left waiting for long periods. These design decisions contribute to a more inclusive gaming environment that supports both new players who are still learning and experienced players who seek competitive challenges.

Accessibility is another key aspect of the system. Unlike many modern FPS games that require high-end hardware, OmniRaid is developed with optimization in mind so that it can run efficiently on mid-range systems without compromising gameplay quality. This approach increases the potential user base and makes the game more practical for a wider audience, especially in environments where high-performance gaming setups are not easily available.

From a development perspective, the project demonstrates the practical application of modern game development technologies and methodologies. It integrates Unity for game design [4] and rendering, Photon Fusion for real-time multiplayer networking, and PlayFab for backend services such as player data management and progression tracking. The combination of these technologies reflects a well-structured and scalable system architecture capable of handling real-time interactions and persistent data efficiently.

OmniRaid is not just a game but a complete system that showcases how performance optimization, networking, and user-centered design can be combined to create a reliable multiplayer experience. It highlights the importance of balancing technical implementation with user experience, ultimately resulting in a platform that is both functional and engaging. This makes the project a strong example of how modern tools and design principles can be used to solve real-world challenges in multiplayer game development.

1.1. Brief

The OmniRaid project is designed as a modern online multiplayer first-person shooter (FPS) game that connects players through a fast and competitive real-time gaming environment. The primary aim of the project is to address common issues found in existing multiplayer FPS games, such as high latency, unbalanced matchmaking, inconsistent gameplay, and limited accessibility for users with mid-range systems.

OmniRaid provides a performance-focused solution by integrating smooth networking, fair matchmaking, and optimized gameplay mechanics into a single unified system. The game ensures continuous and engaging matches by incorporating AI-controlled bots [5], multiple gameplay modes, and responsive controls that enhance the overall player experience. It is designed to maintain a balance between competitive intensity and accessibility, making it suitable for both casual and skilled players. The system is developed using modern game development technologies, including Unity for game design and rendering, Photon Fusion for real-time multiplayer networking, and PlayFab for backend services such as player data management and progression tracking. These technologies enable the game to deliver reliable, scalable, and user-friendly multiplayer experience.

1.1.1. Purpose of Project

The primary purpose of OmniRaid is to develop a high-performance multiplayer first-person shooter (FPS) game that delivers real-time, competitive gameplay through a stable and scalable online infrastructure. The project aims to overcome common challenges in multiplayer gaming, such as network lag, unbalanced matchmaking, and inconsistent synchronization, by implementing efficient networking using Photon Fusion and reliable backend services through PlayFab. It also seeks to enhance player experience by providing smooth controls, fair competition, and secure session management, allowing players to engage in fast-paced matches with consistent performance and minimal disruptions.

1.1.2. Scope of the System

The scope of OmniRaid spans multiple platforms, primarily targeting PC players, with potential future expansion to consoles. The system encompasses the core components necessary for a modern multiplayer FPS experience, including a robust game client developed in Unity, a scalable networking framework powered by Photon Fusion, and a cloud-based backend managed through PlayFab.

OmniRaid handles several key functionalities essential for competitive multiplayer gaming. These include real-time matchmaking, player profile management, in-game statistics tracking, weapon and loadout customization, team-based combat modes, and seamless synchronization of player actions across networked sessions. Additionally, the system incorporates anti-cheat mechanisms and latency optimization to ensure fair and responsive gameplay, even under varying network conditions. By integrating these components, OmniRaid aims to provide a complete multiplayer FPS solution that delivers fast-paced, immersive gameplay while maintaining technical stability and scalability. The system is designed not only to support current gameplay needs but also to accommodate future enhancements, such as new maps, game modes, and cross-platform connectivity, thereby ensuring a long-term, engaging experience for players.

1.1.3. Technologies Used

OmniRaid is developed using **Unity** as the primary game engine, providing a versatile and high-performance environment for building immersive 3D FPS gameplay. **Photon Fusion** [1] handles networking, enabling real-time multiplayer interactions with minimal latency and reliable synchronization across clients. For backend services, **PlayFab** [6] manages player accounts, matchmaking, and data persistence, offering scalable and secure infrastructure suitable for online competitive gaming. Together, these technologies create a robust framework that supports responsive gameplay, efficient server communication, and seamless scalability, forming a solid foundation for a modern multiplayer FPS experience.

1.1.4. System Overview

OmniRaid operates as a fully integrated multiplayer FPS platform designed to deliver fast-paced, real-time combat experiences for online players. The system caters to different user roles, ensuring both gameplay engagement and backend efficiency.

Players can create accounts, customize their avatars, join multiplayer sessions, select weapons and loadouts, and participate in competitive matches across diverse maps. They can also track performance statistics, achievements, and in-game progression through a structured player dashboard.

Game hosts and session managers can access tools for matchmaking, lobby creation, and monitor ongoing matches to ensure balanced and fair gameplay. The system dynamically pairs players based on skill levels and network conditions, minimizing latency and maintaining performance consistency. The admin panel oversees player account management, game configuration, leaderboard maintenance, and anti-cheat enforcement. It also monitors server health, session stability, and overall system performance, ensuring a secure, smooth, and scalable multiplayer environment for all participants.

1.1.5. Outcome of the Project

The project delivers a fully functional multiplayer FPS game that provides a seamless, competitive, and engaging online experience for players. The integrated system ensures smooth real-time gameplay, balanced matchmaking, and responsive controls, significantly enhancing player satisfaction. Through optimized networking with Photon Fusion and robust backend management via PlayFab, the system reduces latency, minimizes connection disruptions, and maintains consistent performance across diverse network conditions. Players benefit from clear progression tracking, achievement systems, and competitive leaderboards, which foster long-term engagement and replayability.

The outcome is a stable, secure, and scalable multiplayer environment where players can enjoy fast-paced combat without technical hindrances, enabling fair competition and a highly interactive gaming experience that meets modern multiplayer FPS standards.

1.2. Relevance to Course Module

The OmniRaid project is closely aligned with the core subjects covered in the Bachelor of Science in Software Engineering program. Each phase of the development process, from requirement analysis to system design, implementation, and testing, demonstrates practical application of theoretical knowledge acquired during the degree.

The project illustrates the integration of multiple key concepts, including software engineering principles for system design, computer networks for real-time multiplayer synchronization, database management for player profiles and leaderboards, and object-oriented programming in Unity for game mechanics implementation.

By developing a complete multiplayer FPS system with Unity, Photon Fusion, and PlayFab, the project showcases how academic learning in programming, networking, and system architecture [7] can be transformed into a functional, interactive, and scalable gaming platform. The following subheadings detail the contribution of major course modules to specific aspects of OmniRaid's development.

1.2.1. Software Engineering

Software Engineering principles were integral in structuring and developing the OmniRaid multiplayer FPS game. Key concepts such as requirement analysis, system modeling, documentation, and adherence to the Software Development Life Cycle (SDLC) [8] were directly applied throughout the project.

The development process followed structured practices, including creation of detailed use case diagrams for player interactions, design diagrams for game mechanics and network synchronization, and systematic testing to ensure smooth real-time multiplayer performance.

Understanding both functional requirements, like matchmaking and combat mechanics, and non-functional requirements, such as latency optimization and scalability, allowed the project to be implemented in an organized and efficient manner. By applying these core Software Engineering concepts, OmniRaid was built as a robust, maintainable, and scalable multiplayer system, demonstrating a clear bridge between academic learning and practical game development.

1.2.2. Database System

Database Systems knowledge was crucial in designing and implementing a secure and efficient backend for OmniRaid. The project uses PlayFab's cloud database services to manage player profiles, match histories, leaderboards, in-game statistics, and progression data.

The database design incorporates core concepts such as data relationships, indexing for fast retrieval, data consistency, and integrity, ensuring that player information is stored securely and accessed efficiently during gameplay. Advanced principles, including transactional updates for in-game events and leaderboard synchronization, were applied to maintain accuracy and responsiveness under high concurrent player activity.

By applying database management techniques, OmniRaid provides a reliable and scalable environment where critical game data is consistently maintained, contributing directly to smooth gameplay, accurate progression tracking, and an overall stable multiplayer experience.

1.2.3. Game Development

Game Development knowledge formed the backbone of implementing OmniRaid as a robust multiplayer FPS. Concepts from object-oriented programming, real-time rendering, physics simulation, and interactive design were applied to create an immersive and responsive gaming experience.

Unity was used as the primary game engine, enabling the integration of game mechanics, character controls, weapon systems, and environment interactions. Photon Fusion facilitated real-time networking, allowing players to connect seamlessly in multiplayer sessions with minimal latency. PlayFab managed backend services, including player authentication, matchmaking, and cloud storage for game data.

By applying structured game development practices, including scene design, prefab management, animation integration, and iterative testing, OmniRaid achieves smooth gameplay, balanced mechanics, and scalable multiplayer interactions. This project demonstrates how theoretical knowledge from programming, game design, and networking courses can be effectively translated into a complete, interactive FPS gaming platform.

1.2.4. Object Oriented Programming

Object-Oriented Programming principles were fundamental in structuring OmniRaid's game code. Concepts such as encapsulation, inheritance, and polymorphism were applied to design modular classes for players, weapons, game mechanics, and UI components. Using OOP allowed for maintainable, reusable, and scalable code, enabling efficient development of complex features like character behavior, weapon interactions, and event handling in real-time multiplayer scenarios.

1.2.5. Data Structure and Algorithms

Data Structures and Algorithms knowledge played a critical role in optimizing OmniRaid's performance. Efficient data structures like arrays, lists, dictionaries, and priority queues were used for inventory management, leaderboard ranking, and matchmaking. Algorithms for sorting, searching, and pathfinding ensured fast computation of player statistics, AI behavior, and network synchronization, contributing to a smooth and responsive gaming experience.

1.2.6. Human Computer Interaction

Human-Computer Interaction principles guided the design of OmniRaid's user interface [9] and gameplay experience. Interactive menus, intuitive controls, and clear HUD elements were developed to enhance player engagement and accessibility. Feedback mechanisms, visual cues, and consistent interaction patterns ensured that players can navigate the game seamlessly and respond efficiently during fast-paced matches.

1.2.7. Information Security

Information Security concepts were applied to protect player data and ensure secure multiplayer sessions. Authentication, encrypted communication, and secure cloud storage via PlayFab safeguarded sensitive player profiles, match histories, and progression data. Anti-cheat measures and secure session management were also implemented to maintain fair play and integrity within the multiplayer environment.

1.2.8. Computer Networks

Computer Networks knowledge was crucial for implementing OmniRaid's real-time multiplayer functionality. Photon Fusion facilitated low-latency client-server communication, session synchronization, and matchmaking. Concepts like packet handling, latency optimization, and network reliability ensured that players experience smooth and consistent gameplay across different network conditions.

1.2.9. Final Year Project I and II

The Final Year Project courses provided the structured framework, milestones, and academic guidance necessary to plan, design, and implement OmniRaid. These courses ensured that the project followed a systematic development process, including phases such as requirement analysis, design documentation, prototype development, iterative testing, and final demonstration.

Skills acquired throughout the degree, such as programming in C# within Unity, object-oriented design, database management with PlayFab, real-time networking with Photon Fusion, user interface design, and security implementation, were directly applied to develop a cohesive and fully functional multiplayer FPS game.

OmniRaid thus reflects the integration of multiple subjects learned during the Bachelor of Science in Computer Science program, demonstrating how theoretical knowledge can be translated into a practical, interactive, and scalable real-world application. The project showcases a complete combination of game development, system design, networking, and backend management skills honed throughout the academic program.

1.3. Project Background

The idea behind OmniRaid originates from the rapid evolution of the gaming industry [2], particularly in the domain of multiplayer first-person shooter (FPS) games. Over the past decade, online multiplayer gaming has shifted from a casual activity to a highly competitive and immersive experience, where players expect real-time interaction, smooth performance, and fair gameplay. Despite this progress, many existing multiplayer FPS systems continue to face persistent challenges such as network latency, unstable connections, inconsistent synchronization, and unbalanced matchmaking. These issues significantly impact on the overall player experience, especially in fast-paced environments where even minor delays can affect gameplay outcomes.

Traditional implementations of multiplayer systems [10] often struggle to maintain consistency across different network conditions. Players connecting from varied geographical locations experience different levels of latency, resulting in unfair advantages or disadvantages during gameplay. In addition, poor synchronization between clients can lead to issues such as delayed hit registration, inconsistent player movement, and unreliable game states. These technical limitations not only reduce the quality of gameplay but also discourage long-term player engagement in competitive scenarios.

With the continuous advancement of game development technologies and networking frameworks, there is an increasing demand for systems that can provide stable and responsive multiplayer experiences. Modern players expect seamless connectivity, quick matchmaking, and reliable performance regardless of their network environment. This shift has created a need for solutions that integrate efficient networking, structured backend systems, and optimized game logic into a unified platform. OmniRaid is conceptualized as a response to this need, aiming to provide a smooth and competitive multiplayer FPS experience through the use of modern tools and technologies.

The project idea was further refined by analyzing common problems faced by players in existing online FPS games. One of the most significant issues is high latency, which disrupts real-time interaction and affects player performance. In competitive gameplay, delayed responses can result in missed actions, unfair eliminations, and overall frustration. Additionally, ineffective matchmaking systems often group players with significantly different skill levels, leading to unbalanced matches that reduce both competitiveness and enjoyment. OmniRaid addresses these concerns by focusing on optimized networking and structured matchmaking to create a more balanced and responsive gameplay environment.

Another key factor influencing the project is the management of player data and game progression. In many multiplayer systems, inefficient backend handling can lead to data inconsistency, delayed updates, or even loss of player progress. This directly impacts user trust and long-term engagement. OmniRaid incorporates a cloud-based backend system to manage player profiles, match histories, leaderboards, and in-game statistics in a consistent and secure manner. This ensures that player progress is accurately maintained and readily accessible, enhancing the overall reliability of the system.

Furthermore, the need for fair and competitive gameplay played an important role in shaping the project. Multiplayer games require a balanced environment where players can compete on equal footing. Unoptimized systems often fail to maintain this balance due to irregular matchmaking or inconsistent game performance. OmniRaid integrates structured gameplay mechanics and

matchmaking strategies to ensure that players are paired appropriately, promoting fairness and improving the competitive experience.

The project is also influenced by the importance of scalability in modern multiplayer systems. As the number of concurrent players increases, the system must be capable of handling multiple sessions without compromising performance. By utilizing technologies such as Photon Fusion for real-time networking and PlayFab for backend services, OmniRaid is designed to support scalable multiplayer interactions while maintaining stability and responsiveness.

In addition to technical considerations, user experience plays a vital role in the development of the project. A well-designed interface, responsive controls, and clear feedback mechanisms are essential for engaging players in a fast-paced FPS environment. OmniRaid focuses on providing intuitive controls and interactive gameplay elements that allow players to react quickly and navigate the system efficiently during matches.

Overall, OmniRaid is designed as a modern multiplayer FPS solution that addresses the limitations of existing systems by combining real-time networking, efficient backend integration, and structured gameplay design. The project reflects the practical application of current technologies to create a stable, responsive, and engaging gaming platform. By focusing on performance optimization, fair competition, and reliable data management, OmniRaid aims to deliver a high-quality multiplayer experience that aligns with the expectations of today's gaming community.

1.4. Literature Review

The review of existing multiplayer FPS games highlights that modern gaming platforms provide advanced features such as real-time combat, matchmaking systems, ranking mechanisms, and player progression. These features have significantly enhanced user engagement by creating competitive and interactive environments where players can continuously improve their skills and track their performance. Popular titles in the industry have established high standards in terms of graphics, gameplay mechanics, and online connectivity, shaping user expectations for smooth and responsive multiplayer experiences. However, despite these technological advancements, several limitations still persist, particularly in areas such as performance consistency, matchmaking fairness, and backend reliability, which directly affect the overall quality of gameplay.

One of the most critical challenges observed in multiplayer FPS games is network latency and synchronization issues. Real-time interaction is the core of FPS gameplay, and even minor delays can disrupt the accuracy of player actions. In many existing systems, players experience lag, delayed hit registration, and inconsistent movement synchronization due to fluctuations in network conditions. These issues become more prominent in cross-region gameplay, where differences in latency can create unfair advantages or disadvantages among players. As a result, the competitive integrity of matches is compromised, leading to frustration and reduced user satisfaction, especially among players who rely on precision and timing.

Another important limitation is related to the effectiveness of matchmaking systems. Although many games implement ranking and skill-based algorithms, they often fail to consistently ensure balanced matches. Players are sometimes grouped with opponents who have significantly higher or lower skill levels, resulting in either overly difficult or unchallenging gameplay scenarios. This imbalance not only affects the enjoyment of individual matches but also impacts long-term player retention, as users may lose interest when the competitive environment is not fair or engaging.

Additionally, some systems do not adequately consider factors such as network quality during matchmaking, which further contributes to inconsistent gameplay experiences.

In addition to gameplay-related challenges, backend system limitations also play a significant role in affecting overall performance. Many multiplayer platforms rely on backend services to manage player data, leaderboards, and progression systems. However, inefficient data handling or delayed synchronization can lead to inconsistencies such as inaccurate rankings, delayed updates in player statistics, or temporary loss of progress. These issues reduce user trust in the system and highlight the need for more reliable and scalable backend integration that can handle real-time updates efficiently. To clearly highlight these gaps, a comparative analysis is presented between well-known multiplayer FPS games and the proposed OmniRaid system. The table below outlines the key weaknesses identified in existing platforms and the corresponding solutions offered by OmniRaid. This comparison provides a structured understanding of how the proposed system addresses the limitations found in current implementations and demonstrates the need for a more optimized, integrated, and scalable multiplayer FPS solution.

Table 1.1: Gap Analysis and Proposed Solutions for Existing Multiplayer FPS Games in OmniRaid

Application Name	Weakness	Proposed Project Solution (OmniRaid)
Call of Duty (Multiplayer Mode)	Players often experience latency spikes and inconsistent hit registration during high-load matches.	OmniRaid utilizes Photon Fusion for optimized real-time networking to minimize latency and ensure accurate synchronization of player actions.
Counter-Strike: Global Offensive (CS:GO)	Matchmaking can sometimes pair players with uneven skill levels, affecting competitive balance.	OmniRaid implements structured matchmaking to ensure players are matched based on skill and performance metrics.
PUBG (Multiplayer Mode)	High dependency on network stability, leading to lag and delayed response in real-time combat.	OmniRaid is designed to handle varying network conditions, ensuring smoother gameplay through efficient synchronization techniques.
Battlefield Series	Large-scale multiplayer environments can lead to performance drops and server instability.	OmniRaid focuses on optimized session management and scalable architecture to maintain stable performance across matches.
Apex Legends	Backend delays can sometimes affect real-time updates of player stats and progression.	OmniRaid integrates PlayFab for reliable and real-time data management, ensuring accurate tracking of player progress and statistics.
Valorant	Requires stable network conditions; performance may degrade in high latency environments.	OmniRaid emphasizes low-latency communication and efficient network handling to provide consistent gameplay across different conditions.

1.5. Analysis from Literature Review

The analysis of the reviewed literature demonstrates how the OmniRaid project aligns with current trends in multiplayer game development and how it effectively addresses the limitations observed in existing FPS systems. The literature highlights key areas such as real-time networking, matchmaking efficiency, backend data management, and gameplay responsiveness as critical components of modern multiplayer games. These findings directly support the core features implemented in OmniRaid and validate the importance of developing a system that ensures smooth, fair, and scalable online gameplay. The comparison between existing solutions and OmniRaid shows that the project is not only aligned with modern gaming standards but is also designed to overcome performance and reliability issues identified in current implementations.

1.5.1. Alignment with Modern Multiplayer Gaming Trends

The literature indicates a significant shift from basic multiplayer implementations to highly optimized, real-time online gaming environments. Modern multiplayer games emphasize low latency, continuous synchronization, and seamless interaction between players. Research highlights that players expect stable connections, responsive controls, and uninterrupted gameplay regardless of network conditions.

OmniRaid aligns with these trends by implementing real-time multiplayer functionality using Photon Fusion, ensuring efficient synchronization of player actions and game states. The system is designed to provide a smooth and responsive gaming experience, reflecting the principles identified in modern multiplayer game development. This alignment demonstrates that the project is built upon established industry trends and addresses the evolving expectations of online players.

1.5.2. Real-Time Networking and Synchronization Analysis

Studies on multiplayer game networking emphasize the importance of minimizing latency and maintaining synchronization between connected players. Many existing systems struggle with delayed updates, inconsistent player positioning, and unreliable hit detection due to inefficient network handling. These issues significantly affect gameplay accuracy in FPS environments.

OmniRaid addresses these challenges by utilizing an optimized networking framework that ensures real-time communication between clients. Player movements, actions, and interactions are synchronized efficiently to maintain consistency across all participants. This approach reflects the networking principles discussed in the literature and ensures that gameplay remains accurate and fair even under varying network conditions.

1.5.3. Comparison with Existing Matchmaking Systems

The literature highlights that matchmaking is a critical component in multiplayer games, as it directly impacts fairness and player satisfaction. Many systems implement basic matchmaking techniques but fail to consistently balance players based on skill level or performance metrics. This results in uneven matches, reducing competitiveness and engagement.

OmniRaid improves upon these limitations by incorporating structured matchmaking mechanisms that aim to group players based on relevant performance factors. This ensures a more balanced gameplay environment where players can compete fairly. The analysis shows that OmniRaid not only adopts the concept of matchmaking from existing systems but enhances its effectiveness to improve overall user experience.

1.5.4. Backend Data Management and Player Progression

Research on [10] highlights the importance of reliable backend infrastructure for managing player data, statistics, and progression. Inefficient data handling can lead to inconsistent updates, inaccurate leaderboards, and potential loss of player progress, which negatively impacts user trust.

OmniRaid integrates PlayFab as a backend solution to securely manage player profiles, match history, and progression data. The system ensures that all player-related information is stored consistently and updated in real time. This implementation reflects the best practices identified in the literature and provides a stable foundation for maintaining long-term player engagement.

1.5.5. Application of Security in Multiplayer Systems

The literature emphasizes that multiplayer platforms must implement security measures to protect player data and maintain fair gameplay. Common concerns include unauthorized access, data manipulation, and cheating, which can compromise the integrity of the system.

OmniRaid incorporates secure authentication and controlled data access through its backend services. In addition, the system considers anti-cheat mechanisms and secure session handling to ensure fair competition among players. This demonstrates alignment with the security practices discussed in the literature and ensures that the system remains reliable and trustworthy.

1.5.6. User Experience in FPS Gameplay

Studies on game design highlight the importance of user experience, particularly in FPS games where fast decision-making and precise controls are essential. A well-designed interface, clear visual feedback, and responsive controls significantly enhance player engagement.

OmniRaid applies these principles by focusing on intuitive controls, structured UI elements, and responsive gameplay mechanics. Players can interact with the system efficiently, making quick decisions during matches without confusion or delay. This reflects the usability concepts discussed in the literature and ensures an engaging and accessible gaming experience.

1.5.7 Addressing Limitations of Existing Multiplayer FPS Systems

The literature identifies several limitations in current multiplayer FPS implementations, including high latency, poor synchronization, unbalanced matchmaking, and unreliable backend systems. These challenges reduce gameplay quality and limit the scalability of such systems.

OmniRaid is specifically designed to overcome these issues by integrating efficient networking, structured matchmaking, and reliable backend management into a unified system. The project ensures smooth gameplay, accurate data handling, and fair competition, addressing each limitation highlighted in the literature. This analysis confirms that OmniRaid provides a more complete and optimized solution compared to existing implementations.

1.6. Methodology and Software Lifecycle

The development of the OmniRaid project follows a structured and disciplined approach that combines modern software engineering practices with an adaptive Software Development Life Cycle (SDLC) model. Since the system involves multiple interconnected components, including

real-time multiplayer gameplay, backend data management, and user interaction systems, a methodology that supports iterative development and continuous feedback was essential.

To effectively handle the dynamic nature of multiplayer game development, an approach was selected that allows flexibility in design, rapid testing of gameplay features, and continuous improvement based on evaluation. This ensures that all modules, including player mechanics, networking systems, and backend integration, are developed in a controlled and systematic manner. By following a well-defined methodology, the project maintains consistency, reduces development risks, and ensures that both functional and non-functional requirements such as performance, scalability, and responsiveness are achieved.

1.6.1. Methodology

The OmniRaid project adopts the Agile methodology, which focuses on iterative development, incremental delivery, and continuous collaboration. Agile is particularly suitable for game development projects where requirements can evolve during the development process based on testing results and gameplay feedback.

This approach allows the development team to break down the project into manageable components and implement them in stages rather than attempting to build the entire system at once. It also enables frequent evaluation of progress, ensuring that each feature meets the expected functionality and performance standards before moving to the next stage.

1.6.2. Agile Practices Applied

In OmniRaid, Agile practices are implemented through short and focused development cycles known as sprints. Each sprint is dedicated to developing specific features such as player movement, shooting mechanics, multiplayer synchronization, user interface elements, and backend connectivity.

At the end of each sprint, the implemented features are tested and evaluated to identify issues related to gameplay performance, responsiveness, or system integration. Feedback is continuously incorporated to refine and improve the system. This iterative process ensures that the project evolves gradually, with each version becoming more stable and optimized.

Regular progress tracking and planning activities help in identifying challenges early and allow timely adjustments in development priorities, ensuring that the project remains aligned with its objectives.

1.6.3. Software Development Life Cycle (SDLC) Model

The project follows the Scrum model within the Agile SDLC framework due to its effectiveness in managing complex, multi-module systems. Scrum divides the project into time-bound sprints, during which specific tasks are planned, developed, and tested.

Each sprint focuses on delivering a functional component of the system, such as implementing multiplayer connectivity using Photon Fusion or integrating backend services through PlayFab. This structured approach ensures that development remains organized and that each module is properly validated before integration into the complete system.

1.6.4. Incremental Delivery

Incremental delivery is a key aspect of the Agile approach followed in OmniRaid. The system is developed in stages, where each sprint delivers a working feature that can be tested independently.

For example, initial stages focus on implementing core gameplay mechanics such as player movement and shooting, followed by integration of multiplayer networking, and later the addition of backend services for data management. This step-by-step approach allows continuous validation of each component, ensuring that issues are identified and resolved early in the development process.

It also enables better control over system complexity and ensures that the final product is stable and fully functional.

1.6.5. Development Tools and Environment

OmniRaid is developed using a set of modern tools and technologies that support real-time multiplayer functionality and scalable system design. Unity is used as the primary game engine for implementing gameplay mechanics, graphics, and user interaction.

Photon Fusion is utilized for real-time networking, enabling efficient synchronization of player actions and game states across multiple clients. PlayFab serves as the backend platform for managing player data, authentication, leaderboards, and cloud storage.

These tools are selected based on their ability to support modular development, performance optimization, and seamless integration between different system components.

1.6.6. Testing and Quality Assurance

Testing [11] is integrated throughout the development process to ensure system reliability, performance, and functionality. Each feature is tested during its respective sprint to identify issues related to gameplay mechanics, networking performance, and backend integration.

Different levels of testing are performed, including unit testing for individual components, integration testing for system modules, and gameplay testing to evaluate user experience. Special attention is given to real-time multiplayer aspects such as synchronization accuracy, latency handling, and session stability.

Quality assurance practices also include continuous evaluation and refinement to ensure that the system meets both functional requirements and performance expectations.

1.6.7. Advantages of the Selected Methodology and SDLC

The Agile-Scrum approach provides several advantages for the development of OmniRaid. It allows flexibility in handling changing requirements, supports continuous improvement through feedback, and ensures early detection of issues.

The incremental development process reduces risk by validating each component before full system integration. It also enhances productivity by dividing the project into manageable tasks and maintaining clear development priorities.

Overall, this methodology ensures that OmniRaid is developed as a stable, scalable, and high-quality multiplayer FPS system, capable of delivering smooth and engaging gaming experience.

Chapter 2

PROBLEM DEFINITION

2. Problem Definition

The rapid evolution of the gaming industry, particularly in the domain of multiplayer first-person shooter (FPS) games, has created a strong demand for highly responsive, competitive, and real-time interactive systems. Players today expect seamless gameplay experiences where actions are executed instantly, outcomes are accurate, and competition is fair. However, despite advancements in game development engines and networking technologies, many existing multiplayer FPS systems still struggle to meet these expectations consistently. These limitations highlight a significant gap between user expectations and the actual performance of current systems.

One of the most critical problems in multiplayer FPS environments is the issue of network latency and inconsistent synchronization. FPS games rely heavily on precise timing and accurate interaction between players. Even a slight delay in transmitting player actions can lead to serious inconsistencies such as delayed hit registration, incorrect positioning of players, and desynchronized game states. These issues are particularly noticeable in fast-paced combat scenarios where milliseconds can determine the outcome of an encounter. Players experiencing higher latency often face disadvantages, making gameplay unfair and frustrating. This inconsistency reduces the overall reliability of the system and directly impacts the competitive nature of the game.

Another major challenge lies in maintaining consistent performance across different network conditions. Players connect to multiplayer games from various geographical locations using networks with varying speeds and stability. Many existing systems are not fully optimized to handle such diversity, resulting in fluctuating gameplay quality. Some players may experience smooth performance, while others encounter lag spikes, packet loss, or disconnections. This uneven experience creates an imbalance in gameplay and reduces user satisfaction, as the system fails to provide a uniform environment for all participants.

In addition to networking challenges, ineffective matchmaking systems further contribute to the problem. Matchmaking plays a crucial role in ensuring fair competition by grouping players based on their skill level, experience, or performance metrics. However, in many existing implementations, matchmaking algorithms are either too simplistic or not properly optimized. As a result, players are often matched with opponents who are significantly stronger or weaker than them. This leads to unbalanced matches where either the challenge is too high or the gameplay becomes uninteresting due to lack of competition. Such experiences discourage players from continued participation and negatively affect long-term engagement.

Backend system limitations also represent a significant issue in multiplayer FPS development. These systems are responsible for managing critical data such as player profiles, match history, statistics, and progression. In many cases, inefficient backend integration leads to delayed updates, inconsistencies in stored data, and unreliable leaderboard rankings. For example, a player's performance in a match may not be immediately reflected in their statistics, or leaderboard positions may not update correctly. These inconsistencies reduce user trust in the system and highlight the need for a more reliable and structured approach to data management.

Another important aspect of the problem is the lack of a unified and well-integrated system architecture. Many multiplayer implementations treat gameplay mechanics, networking, and backend services as separate components without ensuring seamless integration between them. This fragmented design approach can lead to synchronization issues, increased system complexity,

and difficulty in maintaining performance under load. When multiple components fail to interact efficiently, it becomes challenging to deliver a smooth and stable gaming experience. This lack of integration is particularly problematic in multiplayer FPS games, where real-time coordination between all system components is essential.

Scalability is another critical concern that is often overlooked in smaller or less optimized systems. As the number of concurrent players increases, the system must be capable of handling multiple sessions without degradation in performance. Many existing solutions are not designed with scalability in mind, leading to server overload, increased latency, and reduced stability during peak usage. This limitation restricts the growth of the system and prevents it from supporting a larger player base effectively. Furthermore, ensuring fairness and integrity in gameplay remains a continuous challenge. [10] must not only provide smooth performance but also maintain a level playing field where outcomes are determined by player skill rather than technical limitations. Issues such as lag advantage, inconsistent synchronization, and lack of proper validation mechanisms can compromise fairness. This reduces the competitive value of the game and discourages players from engaging in long-term gameplay.

The problem, therefore, can be defined as the absence of a well-optimized, integrated, and scalable multiplayer FPS system that can effectively address challenges related to real-time synchronization, latency management, matchmaking efficiency, backend reliability, and system integration. Existing solutions provide partial functionality but fail to deliver a comprehensive system that ensures consistency, fairness, and performance across all aspects of multiplayer gameplay. To address these challenges, there is a need for a system that combines efficient networking, structured matchmaking, reliable backend services, and optimized gameplay mechanics within a single cohesive framework. Such a system should be capable of maintaining real-time accuracy, handling diverse network conditions, managing player data consistently, and supporting multiple concurrent users without compromising performance.

The expected outcome of this project is the development of OmniRaid, a fully functional multiplayer FPS system that resolves the identified limitations through the integration of modern technologies and structured design practices. By utilizing Photon Fusion for real-time networking, the system aims to achieve low-latency communication and accurate synchronization of player actions. PlayFab is used to provide a reliable backend infrastructure for managing player data, statistics, and progression in a secure and scalable manner.

Additionally, OmniRaid incorporates structured matchmaking mechanisms to ensure fair competition among players. The system is designed to create balanced matches by considering relevant performance factors, thereby improving gameplay quality and user satisfaction. The integration of gameplay, networking, and backend components ensures that all parts of the system work cohesively, reducing inconsistencies and improving overall performance.

Ultimately, OmniRaid aims to deliver a stable, responsive, and engaging multiplayer FPS experience that meets modern gaming standards. By addressing the core problems identified in existing systems, the project provides a practical solution that enhances gameplay fairness, improves system reliability, and supports long-term player engagement. This outcome not only fulfills the technical objectives of the project but also demonstrates the effective application of software engineering and game development principles in solving real-world challenges in multiplayer gaming.

2.1. Problem Statement

The multiplayer FPS gaming environment faces several challenges due to network latency, inconsistent synchronization, unbalanced matchmaking, and inefficient backend data management. Players often experience delays in real-time interactions, inaccurate hit detection, and unstable gameplay under varying network conditions, which negatively impacts performance and fairness. In many systems, matchmaking mechanisms fail to ensure balanced competition, resulting in mismatched players and reduced engagement. Additionally, unreliable handling of player data, including statistics, progression, and leaderboards, leads to inconsistencies and affects user trust in the system.

The OmniRaid project addresses these issues by developing an integrated multiplayer FPS platform that combines real-time networking, structured matchmaking, and reliable backend services. It ensures accurate synchronization of player actions, low-latency communication, and consistent gameplay performance. The system also provides secure and scalable management of player data, including profiles, match history, and leaderboards. By integrating gameplay, networking, and backend components into a unified system, OmniRaid aims to enhance gameplay quality, ensure fair competition, and deliver a stable and engaging multiplayer experience.

2.2. Deliverable and Development Requirements

2.2.1 Deliverables

The primary deliverables of the OmniRaid project focus on developing a complete, functional, and performance-optimized multiplayer gaming system. These deliverables are designed to cover all essential components required for a modern online first-person shooter, including gameplay systems, networking infrastructure, user interfaces, and backend services. Each deliverable contributes to creating a seamless, engaging, and competitive gaming experience while ensuring scalability, reliability, and usability for players.

2.2.1.1.PC Game Application (Client-Side Game)

The main deliverable of the OmniRaid project is a fully developed PC-based multiplayer FPS game built using Unity. This application serves as the primary interface through which players interact with the game environment. It includes core gameplay features such as player movement, shooting mechanics, weapon handling, and real-time combat interactions. The game provides multiple modes like Team Deathmatch, Free-for-All, and Capture the Flag, ensuring variety and engagement for different types of players.

The client application is designed with an intuitive user interface, including HUD elements such as health indicators, ammo count, mini-map, and kill feed. It also integrates smooth animations, optimized graphics, and responsive controls to enhance the overall user experience. Special attention is given to performance optimization so that the game runs efficiently on mid-range systems without compromising visual quality.

2.2.1.2. Multiplayer Networking System

A critical deliverable of the project is the real-time multiplayer networking [12] system powered by Photon Fusion. This system enables players from different locations to connect and participate in matches simultaneously with minimal latency. It handles player synchronization, state replication, and communication between clients and the server.

The networking system implements advanced techniques such as client-side prediction and lag compensation to ensure smooth gameplay even under varying network conditions. It also includes matchmaking functionality that pairs players based on skill level and connection quality, creating balanced and competitive matches. This deliverable plays a vital role in achieving the project's goal of providing a lag-free and responsive multiplayer experience.

2.2.1.3. AI Bot System

The OmniRaid project also delivers an intelligent AI [5] bot system designed to maintain gameplay continuity and balance. These bots are capable of simulating human-like behavior, including movement, target detection, decision-making, and combat strategies. They are integrated into matches to fill empty player slots, ensuring that games remain active even when the number of real players is low.

The AI system includes adjustable difficulty levels, allowing bots to adapt to different player skill levels. This not only enhances the gaming experience for beginners but also provides a consistent challenge for experienced players. The implementation of AI [5] bots ensures that the game remains engaging and playable in both online and semi-offline scenarios.

2.2.1.4.Backend and Player Data Management System

Another key deliverable is the backend system implemented using PlayFab, which manages all persistent player data. This system handles player authentication, profile management, leaderboards, and progression tracking. It ensures that player achievements, experience points (XP), and unlocked items are securely stored and accessible across sessions.

The backend also supports real-time data synchronization and cloud saving, allowing players to resume their progress seamlessly. Additionally, it provides leaderboard functionality that ranks players based on performance, encouraging competition and long-term engagement. This deliverable is essential for maintaining data integrity, scalability, and overall system reliability.

2.2.1.5.User Interface and Experience System

The project includes a comprehensive user interface (UI) system that enhances usability and player interaction. This deliverable covers all visual and interactive components, including the main menu, lobby interface, settings panel, inventory system, and post-match result screens. The UI is designed to be clean, responsive, and easy to navigate, ensuring that players can access features such as matchmaking, customization, and statistics without confusion.

In addition to functionality, the UI system focuses on providing visual feedback through animations, highlights, and notifications. This improves the overall user experience by making the game more immersive and engaging. The interface design follows modern gaming standards, ensuring consistency and accessibility for both new and experienced players.

2.2.1.6.Game Maps and 3D Assets

The final deliverable includes a collection of optimized game maps and 3D assets created using Blender. These maps are designed with different themes such as urban, desert, and sci-fi environments to provide visual variety and strategic gameplay opportunities. Each map is carefully optimized to ensure smooth performance while maintaining high visual quality.

The assets include player models, weapons, environmental objects, and animations, all integrated seamlessly into the game engine. The design of these assets focuses on both aesthetics and

functionality, ensuring that they contribute to gameplay mechanics such as cover, visibility, and movement. This deliverable plays a significant role in enhancing immersion and overall game quality.

2.2.2. Development Requirements

The Development requirements of OmniRaid define the complete technological foundation needed to build, deploy, and maintain a high-performance online multiplayer FPS game. Since the project focuses heavily on real-time interaction, low latency, and smooth gameplay, the selection of tools and technologies is carefully aligned with performance optimization, scalability, and user experience. The system is designed using a hybrid architecture where the client-side game logic, networking layer, and backend services work together seamlessly to deliver a responsive and competitive gaming environment.

Unity

At the core of the development lies the Unity game engine, which is used to design and implement the entire gameplay experience. Unity provides a powerful environment for handling 3D rendering, physics simulation, animation, and user interface development. It allows developers to create immersive environments, responsive controls, and optimized game performance suitable for mid-range and high-end systems. The use of C# scripting within Unity enables structured, object-oriented development, making the system modular and easier to maintain.

Photon Fusion

For real-time multiplayer functionality, OmniRaid integrates Photon Fusion, which plays a critical role in ensuring smooth and synchronized gameplay across multiple players. Photon Fusion uses an authoritative server model with advanced features such as client-side prediction and lag compensation. This ensures that even in varying network conditions, player actions like movement and shooting are accurately synchronized, providing a fair and competitive experience. The networking layer also supports matchmaking, session management, and real-time state replication between players.

PlayFab

To handle persistent data and backend services, the system utilizes PlayFab. PlayFab is responsible for managing player profiles, leaderboards, achievements, and progression systems. It ensures secure cloud-based data storage, allowing players to retain their progress across sessions. Additionally, PlayFab provides authentication mechanisms and analytics support, which helps in monitoring player behavior and improving the overall system performance over time.

Blender

For asset creation and visual design, Blender is used to develop game environments, characters, and props. Blender enables the creation of optimized 3D models and animations that are compatible with Unity, ensuring that the game maintains a balance between visual quality and performance efficiency. This is particularly important for delivering smooth gameplay on systems with moderate hardware specifications.

Visual Studio Code

From a development and coding perspective, tools like Visual Studio Code [13] are used as the primary integrated development environment for writing and managing C# scripts. Additional tools such as Adobe Photoshop and After Effects support UI design and visual effects, enhancing the overall user experience. Finally, the system relies on cloud-based infrastructure for hosting multiplayer services and backend operations. Cloud integration ensures that the game remains accessible to players globally, supports scalability as the number of users increases, and maintains high availability.

2.3. Current System

The analysis of existing systems in the domain of multiplayer first-person shooter (FPS) games focuses on widely used and industry-recognized platforms such as Counter-Strike: Global Offensive (CSGO2), Valorant, and Call of Duty: Warzone. These systems represent modern multiplayer gaming solutions that provide advanced gameplay mechanics, competitive environments, and large-scale player interaction. While these platforms demonstrate strong technical capabilities and user engagement, they also exhibit certain limitations that affect performance, accessibility, and overall gameplay experience.

2.3.1. Counter-Strike: Global Offensive (CSGO2)

Counter-Strike: Global Offensive (CSGO2) is a highly popular multiplayer FPS that emphasizes team-based combat between terrorists and counter-terrorists. The system is known for its balanced gameplay mechanics, precise shooting controls, and competitive ranking system. It also includes multiple maps and customization options such as weapon skins. However, despite its optimized gameplay, CSGO2 may experience latency issues in regions with less optimized servers. This can affect real-time synchronization and reduce gameplay consistency for players in certain locations.

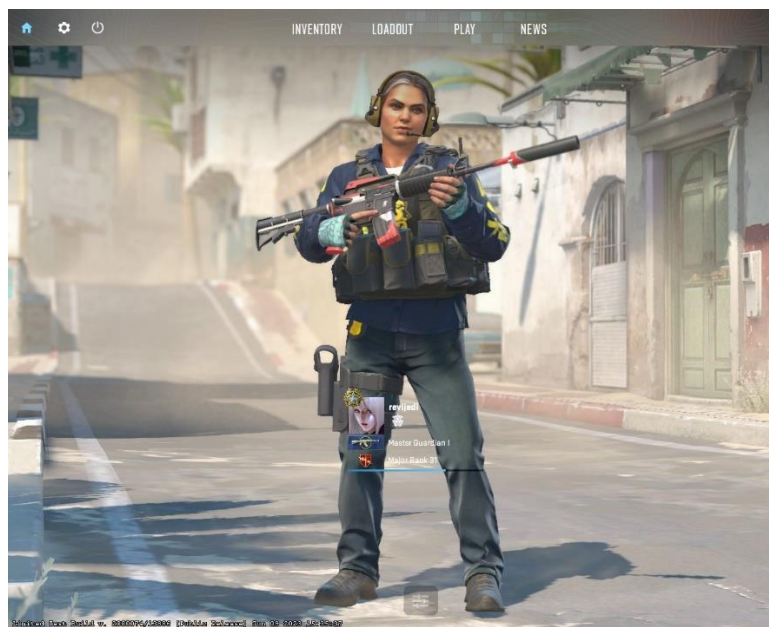


Figure 2.1: CS2 Lobby



Figure 2.2: CS2 Gameplay

2.3.2. Valorant

Valorant is a tactical FPS that integrates traditional shooting mechanics with character-based abilities. It provides high-quality graphics, strong anti-cheat mechanisms, and a well-developed competitive ecosystem. The system emphasizes strategy, coordination, and precision, making it suitable for competitive gameplay environments.

Despite these strengths, Valorant has a steep learning curve, which can make it difficult for new or casual players to adapt. This limits accessibility and reduces its appeal for a broader audience.



Figure 2.3: Valorant Gameplay

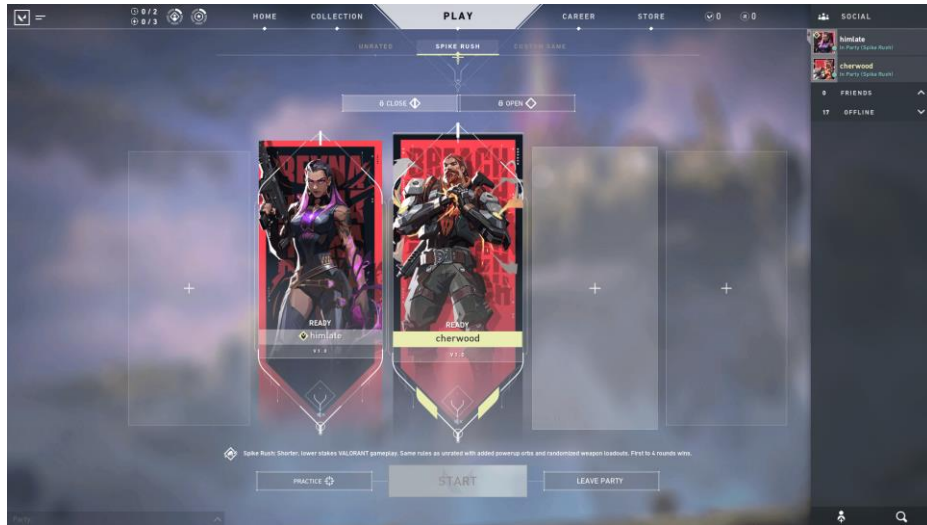


Figure 2.4: Valorant Lobby

2.3.3. Call of Duty: Warzone

Call of Duty: Warzone is a large-scale multiplayer battle royale game that supports a high number of concurrent players within detailed environments. It offers cross-platform compatibility, advanced graphics, and continuous updates to enhance user engagement. The system is designed for fast-paced gameplay and team-based interactions. However, Warzone requires high system resources and large storage capacity, which can act as a barrier for users with mid-range or low-end hardware. This reduces accessibility and limits its usability for a wider range of players.



Figure 2.5: Call Of Duty Warzone Lobby



Figure 2.6: Call Of Duty Warzone Gameplay

2.3.4. Comparative Analysis of Existing Systems

To better understand the strengths and limitations of these systems, a comparison is presented in the following table:

Table 2.2: Comparison of Existing Multiplayer FPS Games with Identified Limitations

Application Name	Key Features	Weakness
Counter-Strike: Global Offensive (CSGO2)	Competitive gameplay, precise gun mechanics, large player base	Latency issues in less optimized regions
Valorant	Tactical gameplay with agent abilities, anti-cheat system, esports focus	Steep learning curve for new players
Call of Duty: Warzone	Large-scale multiplayer, high-quality graphics, cross-platform support	High system requirements and large storage size

2.3.5. Identified Limitations

The following key limitations have been identified in existing systems:

- Network latency affecting real-time gameplay performance
- High skill requirements or complexity reducing accessibility
- High system requirements limiting usage on mid-range devices
- Inconsistent gameplay experience across different regions

2.3.6. Gap Analysis and Motivation for OmniRaid

By analyzing existing multiplayer gaming systems and platforms, several critical gaps have been identified that significantly influence the design and development of OmniRaid. Modern multiplayer games often prioritize highly competitive environments, complex gameplay mechanics, and advanced graphical fidelity. While these features enhance realism and depth, they

also introduce increased system requirements and steep learning curves for new or casual players. As a result, a large portion of potential users, particularly those with mid-range hardware, face accessibility challenges when attempting to engage with such systems.

Furthermore, many existing platforms rely on resource-intensive game engines and high-performance networking architectures, which can lead to performance bottlenecks on less powerful devices. This creates an imbalance where only users with high-end systems can fully experience smooth gameplay, while others encounter frame drops, lag, or reduced graphical quality. Such limitations highlight the need for optimized system design that ensures consistent performance across a broader range of hardware configurations.

Another major challenge observed in current multiplayer environments is network latency and synchronization. Real-time multiplayer systems require efficient data exchange between clients and servers to maintain consistency in gameplay. However, issues such as high latency, packet loss, and poor synchronization can negatively impact player experience, leading to delayed actions, unfair advantages, and overall frustration. These problems are particularly critical in fast-paced game modes where timing and responsiveness are essential.

In addition to networking challenges, matchmaking systems in many existing games often lack efficiency and balance. Traditional matchmaking algorithms may not always consider player skill levels, latency conditions, or regional factors effectively, resulting in unbalanced matches and inconsistent gameplay experiences. This can reduce player engagement and long-term retention, as users may feel either overwhelmed or under-challenged during gameplay sessions.

To address these identified limitations, OmniRaid is designed with a strong focus on accessibility, performance optimization, and efficient multiplayer interaction. The system emphasizes low-latency networking by utilizing modern multiplayer frameworks that support real-time synchronization and minimize communication delays between players. Additionally, the matchmaking system is designed to be fast and balanced, incorporating factors such as player skill level, connection quality, and system performance to ensure fair and engaging matches.

Moreover, OmniRaid aims to provide optimized performance for mid-range systems by carefully managing graphical resources, game logic, and background processes. This ensures that users with moderate hardware configurations can still experience smooth gameplay without significant compromises in quality. The system adopts a balanced approach between competitive gameplay and accessibility, allowing both casual and experienced players to engage effectively.

Unlike many existing solutions that focus solely on high-end competitive gaming, OmniRaid introduces a hybrid model that combines engaging gameplay mechanics with optimized system performance. This approach not only improves user accessibility but also enhances overall player satisfaction by delivering a consistent and responsive multiplayer experience. As a result, OmniRaid is positioned to bridge the gap between performance-intensive gaming platforms and user-friendly, accessible systems, making it suitable for a wider audience.

Chapter 3

REQUIREMENT ANALYSIS

3. Requirement Analysis

The Requirement Analysis chapter provides a clear and structured understanding of what the OmniRaid system must achieve in order to deliver a smooth and engaging multiplayer gaming experience. It acts as a foundation for translating player expectations into well-defined technical requirements by identifying how the system behaves, how players interact with it, and what functionalities are essential for gameplay.

This chapter explains how the OmniRaid system operates, covering both functional and non-functional requirements while also defining the roles of different actors involved in the system. Using the use case diagrams as a reference, the requirements are organized in a way that supports the design and development phases. Through this analysis, the development team gains a better understanding of system goals, limitations, and performance expectations, ensuring that the final game effectively addresses common issues such as lag, poor matchmaking, and unbalanced gameplay.

3.1. Introduction to Requirement Analysis

Requirement Analysis is one of the most important phases in the software development lifecycle, as it forms the base for designing and building the entire system. In the case of OmniRaid, this phase ensures that all gameplay features, system performance needs, and technical constraints are clearly defined before development begins.

The main purpose of this phase is to bridge the gap between what players expect and how the system is implemented technically. It helps in identifying how different components such as gameplay mechanics, multiplayer networking, AI bots, and backend systems will interact with each other. This chapter outlines the system requirements in detail, explaining how players interact with the game, how data flows between components, and what outcomes are expected from each interaction.

3.2. System Overview

The OmniRaid system is designed as a real-time multiplayer FPS platform that connects players within a competitive gaming environment. The system allows players to join matches, compete in different game modes, and interact with other players through a synchronized online network. Players can customize their loadouts, track their progress, and participate in matches that are balanced through matchmaking systems.

The system integrates multiple components including the game client built in **Unity**, real-time networking through **Photon Fusion**, and backend services managed by **PlayFab**. These components work together to provide smooth gameplay, real-time synchronization, and persistent player data storage.

The use case diagrams illustrate how different features such as matchmaking, gameplay actions, AI bot interactions, and progression systems are interconnected. Overall, the system acts as a centralized gaming platform that replaces traditional offline gameplay with a connected, competitive, and performance-optimized multiplayer experience.

3.3. Actors in the System

The OmniRaid system primarily interacts with different types of users, known as actors, each having specific roles within the game environment. These actors perform various actions such as playing matches, managing profiles, or maintaining the system. Understanding these roles is important to ensure that the system meets all functional requirements and provides a smooth user experience.

3.3.1. Player

The player is the main actor of the OmniRaid system and represents the end user of the game. Players interact with the system by joining matches, selecting game modes, and participating in real-time combat. They can customize their weapons and loadouts, track their performance, and progress through ranking systems based on their gameplay.

The system allows players to access features such as matchmaking, leaderboards, and in-game statistics. It ensures that players experience smooth gameplay, fair competition, and responsive controls without technical interruptions.

3.3.2. AI Bot

AI Bots act as secondary actors within the system, designed to simulate real players when needed. Their role is to maintain match continuity by filling empty player slots and ensuring that games remain active even when there are not enough human players available.

These bots are capable of performing actions such as movement, targeting, and combat, making them an important part of the gameplay system. They help in balancing matches and also provide practice opportunities for players.

3.3.3. System / Server

The system or server acts as a controlling actor that manages the overall gameplay environment. It handles matchmaking, synchronizes player actions, processes game logic, and ensures fair gameplay through mechanisms such as lag compensation and anti-cheat systems.

The server is responsible for maintaining real-time communication between players and ensuring that all game events are processed accurately and efficiently.

3.4. Usecase Diagrams

Use case diagrams provide a clear and structured representation of how different users interact with the *OmniRaid* system. They help define the functional boundaries of the game by illustrating what actions each actor—such as players, AI bots, and the system/server—can perform within the environment. Through these diagrams, it becomes easier to understand how core features like matchmaking, gameplay actions, profile management, and progression systems are connected and how users navigate between them.

In the context of OmniRaid, the use case diagram highlights key interactions such as joining a match, selecting game modes, engaging in combat, customizing loadouts, and viewing leaderboards. It also reflects system-level processes like matchmaking, synchronization, and AI bot management, which operate in the background to support gameplay.

Use Case Diagram - Menu System

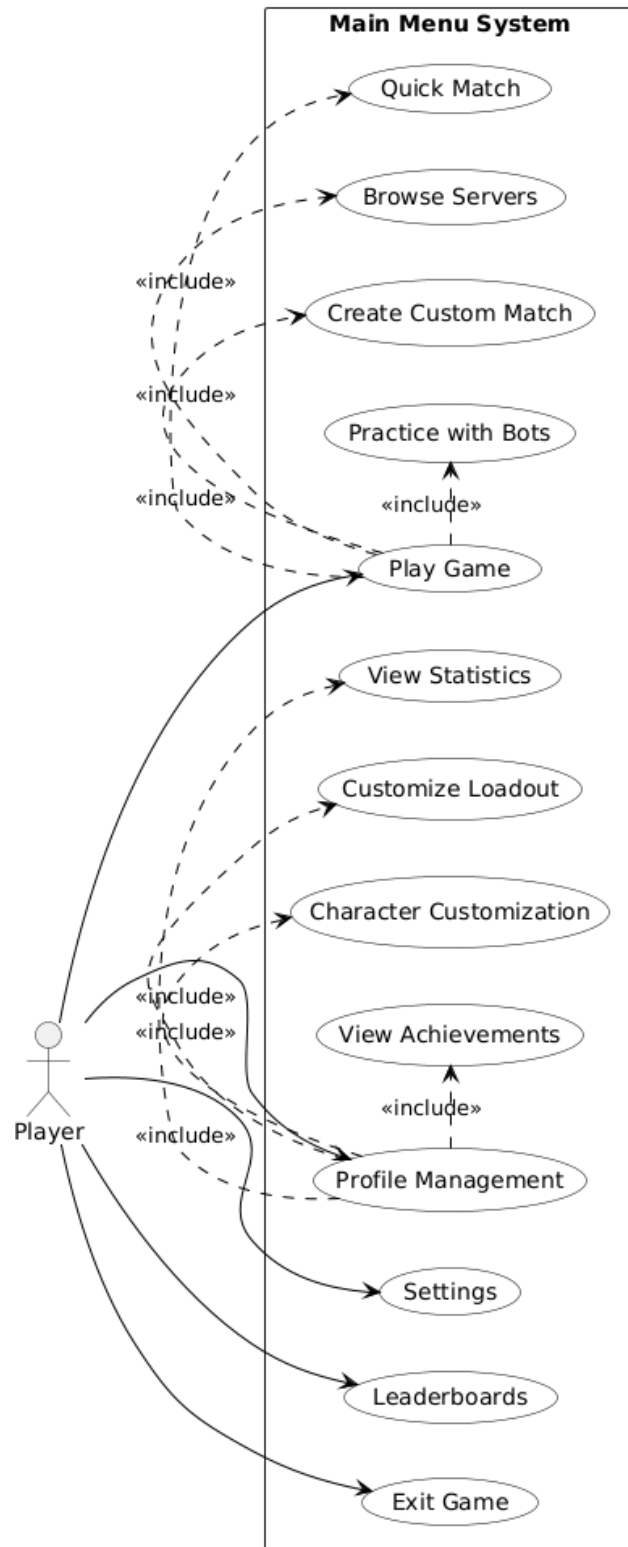


Figure 3.7: OMNIRAID Menu System

Use Case Diagram - Gameplay System

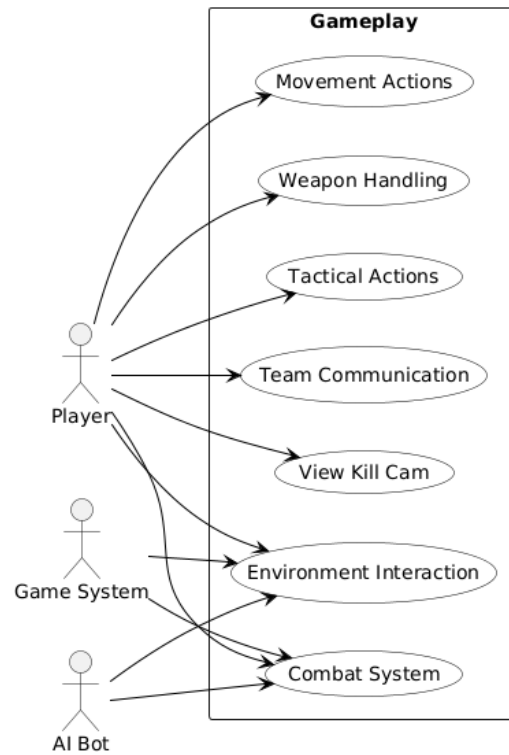


Figure 3.8: OMNIRAID Gameplay

Use Case Diagram - AI Bot System

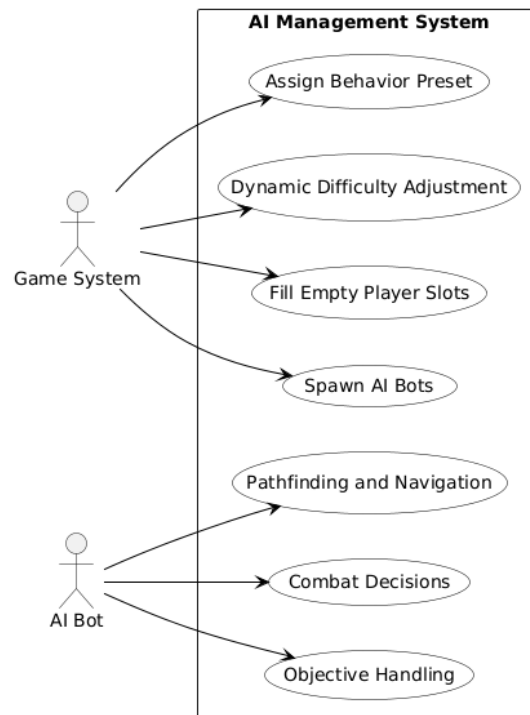


Figure 3.9: OMNIRAID AI Management System

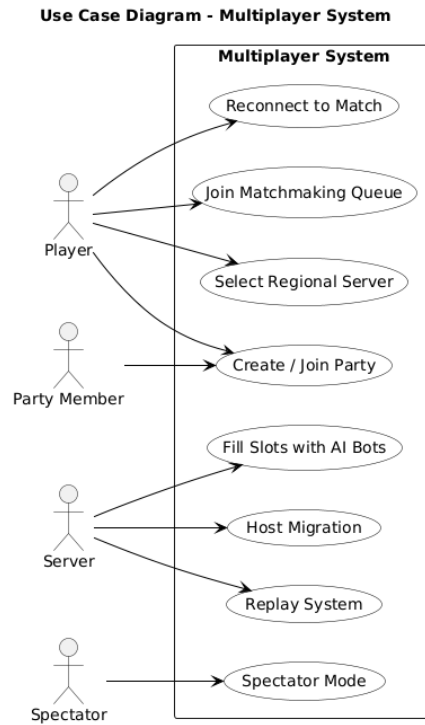


Figure 3.10: OMNIRAID Multiplayer System

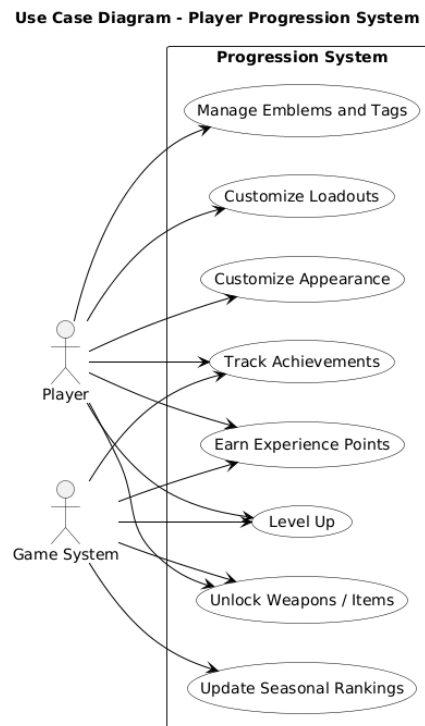


Figure 3.11: Player Progression System

3.5. Use Case Descriptions

3.5.1. AI System

Table 3.3: UC-1

Use Case ID	UC-1
Use Case Name	Access Main Menu
Actors	Player
Description	The player launches the game and navigates through the main menu to select gameplay options, customize their profile, or configure settings.
Trigger	Player starts the game or returns to main menu.
Preconditions	Game client must be running.
Postconditions	Player's access chosen submenu or exit the game.
Normal Flow	• Player selects "Play," "Profile," "Settings," "Leaderboards," or "Exit Game." • System loads and displays the corresponding menu screen.
Alternative Flows	If the player presses "Back," return to the main menu.
Exceptions	Missing assets or network failure while loading menus.
Business Rules	The menu must respond within 2 seconds to user input.
Assumptions	The player has valid local save or profile data.

Table 3.4: UC-1.1

Use Case ID	UC-1.1
Use Case Name	Play Game
Actors	Player
Description	The player selects gameplay options to start a match.
Trigger	Player selects "Play" from main menu.
Preconditions	Players must be logged in or playing offline.
Postconditions	Match session is initialized.
Normal Flow	• Player chooses between Quick Match, Browse Servers, Custom Match, or Practice with Bots. • System loads the selected mode. • Player joins or creates a session.
Alternative Flows	Connection failure during matchmaking.
Exceptions	No available servers; insufficient permissions.
Business Rules	Matchmaking should prioritize low latency servers.
Assumptions	Internet connectivity is available for online modes.

Table 3.5: UC-2

Use Case ID	UC-2
Use Case Name	Manage Profile
Actors	Player
Description	Allows players to view their statistics, achievements, and customize loadouts or character.
Trigger	Player selects "Profile" from the main menu.
Preconditions	Player is authenticated.
Postconditions	Profile data is displayed and may be updated.
Normal Flow	• Player opens the profile section. • System retrieves and displays profile data. • Player navigates to Statistics, Loadout, Customization, or Achievements.

Alternative Flows	Data retrieval failure.
Exceptions	Network timeout during data fetch.
Business Rules	All data must be synced from the player's server account.
Assumptions	Profile data exists on the database.

Table 3.6: UC-3

Use Case ID	UC-3
Use Case Name	Configure Settings
Actors	Player
Description	The player adjusts graphical, audio, control, and network settings.
Trigger	Player selects "Settings."
Preconditions	Game is running.
Postconditions	Settings are saved and applied.
Normal Flow	• Player selects desired category (Graphics, Audio, Controls, Network). • Player adjusts values. • System applies and saves preferences.
Alternative Flows	Player restores default settings
Exceptions	Player restores default settings
Business Rules	Settings changes must not disrupt ongoing session.
Assumptions	Player has sufficient hardware capability.

Table 3.7: UC-4

Use Case ID	UC-4
Use Case Name	View Leaderboards
Actors	Player
Description	Displays player rankings globally, regionally, or among friends.
Trigger	Player selects "Leaderboards."
Preconditions	Active internet connection.
Postconditions	Rankings displayed.
Normal Flow	• System retrieves leaderboard data. • Player switches between Global, Friends, and regional views
Alternative Flows	Player not connected → Show offline message.
Exceptions	Data server unavailable.
Business Rules	Leaderboards update every 5 minutes.
Assumptions	Database contains valid ranking data.

3.5.2. Gameplay

Table 3.8: UC-5

Use Case ID	UC-5
Use Case Name	Perform Gameplay Actions
Actors	Player, Game System
Description	Players perform movement, combat, and interaction actions during a match.
Trigger	Match starts.
Preconditions	Player is spawned in a valid location.
Postconditions	Game state updates accordingly.
Normal Flow	• Player performs movement and shooting actions.

	• System updates positions, health, and weapon states. • Player interacts with environment or communicates with team. • System synchronizes updates across all clients.
Alternative Flows	Player switches weapon or throws equipment.
Exceptions	Connection loss during action execution.
Business Rules	Latency compensation must ensure fair hit registration.
Assumptions	Stable network connection.

Table 3.9: UC-9

Use Case ID	UC-6
Use Case Name	Engage in Combat
Actors	Player, Game System, AI Bot
Description	Handles firing, hit detection, and damage calculation.
Trigger	Player or AI fires a weapon.
Preconditions	Weapon is loaded; target is in range.
Postconditions	Health and status of entities updated.
Normal Flow	• Player fires weapon. • System detects hit using latency compensation. • System applies damage based on hit location and armor. • If target health $\leq 0 \rightarrow$ player gets kill credit.
Alternative Flows	Missed shot or partial hit.
Exceptions	Desync between clients and server.
Business Rules	Kill cam must replay from shooter's perspective.
Assumptions	Server has correct player state synchronization.

3.5.3. Multiplayer System

Table 3.10: UC-7

Use Case ID	UC-7
Use Case Name	Join Matchmaking Queue
Actors	Player, Server
Description	The player enters matchmaking for online multiplayer sessions.
Trigger	Player selects "Quick Match."
Preconditions	Player connected to the internet.
Postconditions	Player joins a match session.
Normal Flow	• Player selects matchmaking mode. • Server finds suitable opponents using skill-based matching. • Player joins session.
Alternative Flows	Match not found $\rightarrow$ Retry or notify player.
Exceptions	Server timeout or unavailable region.
Business Rules	Prioritize low ping and similar skill players.
Assumptions	Regional servers are active.

Table 3.11: UC-8

Use Case ID	UC-8
Use Case Name	Manage Party System

Actors	Player, Party Member
Description	Enables friends to join the same session.
Trigger	Player invites a friend to the party.
Preconditions	Friends list available.
Postconditions	Party created or joined.
Normal Flow	• Player opens party menu. • Player invites friends. • Invited members accept and join.
Alternative Flows	Friend declines invitation.
Exceptions	Invite fails due to connection loss.
Business Rules	Maximum party size = 4 players.
Assumptions	Player's friend list synced.

Table 3.12: UC-9

Use Case ID	UC-9
Use Case Name	Host Migration and Reconnection
Actors	Server, Player
Description	Maintains session continuity when host disconnects or player rejoins.
Trigger	Host leaves or player reconnects.
Preconditions	Active multiplayer session.
Postconditions	Session continues or player rejoins.
Normal Flow	• System detects host disconnection. • Server assigns new host. • Player reconnects to the session.
Alternative Flows	Player fails to reconnect → Match resumes without them.
Exceptions	Host migration timeout
Business Rules	Match must not terminate if 80% of players remain connected.
Assumptions	Redundant server backup available.

3.5.4. AI System

Table 3.13: UC-10

Use Case ID	UC-10
Use Case Name	Manage AI Bots
Actors	Game System, AI Bot
Description	Controls spawning, difficulty, and behavior of AI bots in matches.
Trigger	Player or system starts a match with bots.
Preconditions	AI configuration available.
Postconditions	AI bots act according to preset behaviors.
Normal Flow	• System spawns AI bots. • Assigns difficulty and behavior presets. • AI navigates, attacks, and coordinates with teammates.
Alternative Flows	AI fills empty player slots mid-match.
Exceptions	AI fails to spawn due to resource limit.
Business Rules	AI difficulty adjusts dynamically with player skill
Assumptions	Map supports AI navigation mesh.

3.5.5. Progression System

Table 3.14: UC-11

Use Case ID	UC-11
Use Case Name	Player Progression and Rewards
Actors	Player, Game System
Description	Tracks player XP, level, achievements, and unlocks.
Trigger	Match completion or achievement event.
Preconditions	Player account linked to profile.
Postconditions	Progress updated and saved.
Normal Flow	• System awards XP based on match performance. • Player levels up after reaching XP threshold. • Unlocks items or achievements.
Alternative Flows	Offline mode → progress synced later.
Exceptions	Server data write failure.
Business Rules	XP cannot be earned through idle time.
Assumptions	Database connection available for progression save.

3.6. Functional Requirements

3.6.1. Menu System Requirements

Table 3.6.1.15: FR-1

Identifier	FR-1
Title	Menu System
Requirement	The system should provide a main and in-game menu interface allowing users to access game modes, player profiles, settings, leaderboards, and exit options. Main Menu: Play, Profile, Settings, Leaderboards, Exit. In-Game Menu: Resume, Settings, Change Loadout, View Scoreboard, Report Player, Leave Match.
Source	Game Design Document
Rationale	To provide an intuitive navigation structure that enables players to access all game features efficiently.
Business Rule	Users must be logged in to access Profile and Leaderboards.
Dependencies	UI framework, backend profile and leaderboard services.
Priority	High

3.6.2. Gameplay Requirements

Table 3.6.2.16: FR-2

Identifier	FR-2
Title	Gameplay System

Requirement	The system shall allow players to perform movement, combat, and tactical actions within the game environment. It shall include hit registration, damage calculation, recoil, armor, and kill-cam functionalities. The game shall provide multiple game modes including Team Deathmatch, Free-for-All, Capture the Flag, and Custom Matches, each with specific rules, time limits, and respawn conditions.
Source	Game Mechanics Specification
Rationale	To ensure immersive, fair, and engaging gameplay experiences across various modes.
Business Rule	Players must complete tutorials before accessing online matches.
Dependencies	Physics engine, weapon system, networking system.
Priority	High

3.6.3. Multiplayer Requirements

Table 3.6.3.17: FR-3

Identifier	FR-3
Title	Multiplayer and Networking System
Requirement	The system shall implement skill-based matchmaking, regional server selection, and party functionality. It shall also support host migration, reconnection, spectator, and replay modes to enhance the multiplayer experience.
Source	Online Services Design
Rationale	To ensure fair matchmaking, minimize latency, and improve player retention through reliable online connectivity.
Business Rule	Player accounts must be verified before participating in ranked matches.
Dependencies	Server infrastructure, player authentication system, AI bot system (for backfill).
Priority	High

3.6.4. AI Bot Requirements

Table 3.6.4.18: FR-4

Identifier	FR-4
Title	AI Bot System
Requirement	The system shall include AI-controlled bots capable of dynamic difficulty adjustment, pathfinding, combat, and team coordination. Bots can be configured by difficulty level, behavior preset, and weapon preference. Bots shall automatically fill empty player slots in multiplayer matches.
Source	AI Module Specification
Rationale	To ensure gameplay continuity and balanced matches even when human players are unavailable.
Business Rule	Bots cannot participate in ranked matches.
Dependencies	Navigation mesh system, combat AI module, matchmaking system.
Priority	Medium

3.6.5. Progression System Requirements

Table 3.6.5.19: FR-5

Identifier	FR-5
Title	Player Progression and Customization System
Requirement	The system shall track player experience points, levels, achievements, and seasonal ranks. It shall allow users to customize weapons, characters, emblems, and loadouts through unlocked content
Source	Player Progression Document
Rationale	To increase player engagement, retention, and personalization through rewards and progression.
Business Rule	Unlocks are tied to account progression and cannot be transferred.
Dependencies	Database system, profile management, achievement tracking system.
Priority	Medium

3.7. Non-Functional Requirements

3.7.1. Performance Requirements

The OmniRaid system is designed to deliver a smooth and responsive gameplay experience by meeting specific performance standards. Under optimal conditions, the system aims to maintain a maximum latency of 50 milliseconds to ensure real-time responsiveness during multiplayer interactions. The game is expected to run at a minimum frame rate of 60 FPS on recommended hardware, providing fluid visuals and consistent gameplay. Additionally, map loading times are optimized to remain under 30 seconds, allowing players to quickly enter matches without unnecessary delays. The server operates with a minimum tick rate of 64 Hz, ensuring accurate synchronization of player actions and maintaining fairness in gameplay.

3.7.2. Usability Requirements

Usability is a key focus of the OmniRaid system, ensuring that players can easily interact with the game without confusion or frustration. The interface is designed with intuitive menu navigation, allowing users to access different features such as matchmaking, settings, and profiles efficiently. Clear visual feedback is provided for all in-game actions, helping players understand the results of their interactions instantly. The system also includes accessibility options such as colorblind modes and subtitles to accommodate a wider range of users. A comprehensive tutorial system is integrated to guide new players, while tooltip assistance is available to provide helpful information and improve overall user experience.

3.7.3. Reliability Requirements

The OmniRaid system is built to ensure consistent and dependable performance throughout its operation. It targets a server uptime of 99.5%, minimizing disruptions and ensuring that players can access the game reliably. The system includes automatic crash recovery mechanisms to quickly restore functionality in case of unexpected failures. Data persistence is maintained across sessions, ensuring that player progress, statistics, and achievements are safe and accessible at all times. Additionally, the system is designed to handle high loads efficiently, with graceful degradation strategies that maintain core functionality even under heavy traffic conditions.

3.7.4. Security Requirements

Security is a critical aspect of the OmniRaid system to ensure fair gameplay and protect user data. The system incorporates a robust anti-cheat protection mechanism to prevent hacking and unfair advantages. All network communications are encrypted to safeguard data transmission between clients and servers. Secure authentication protocols are implemented to protect user accounts and prevent unauthorized access. The system also includes protection against common exploits that may affect gameplay integrity. Regular security updates are applied to address emerging threats and maintain a secure gaming environment.

3.7.5. Compatibility Requirements

The OmniRaid system is designed to be compatible with a wide range of hardware and software configurations. It supports both Windows 10 and Windows 11 operating systems, ensuring accessibility for most PC users. The game is built with DirectX 12 compatibility to take advantage of modern graphics capabilities. It also supports common peripherals such as keyboards, mice, and gaming controllers. Additionally, the system accommodates multiple screen resolutions, including ultrawide display support, ensuring a consistent visual experience across different setups.

3.7.6. Scalability Requirements

Scalability is an important consideration in the design of OmniRaid, allowing the system to grow and adapt over time. The current system supports up to 10 players per match, ensuring balanced gameplay while maintaining performance. The architecture is designed to be expandable, enabling the addition of new game modes and features in future updates. A modular design approach is used, allowing developers to integrate new components without affecting existing functionality. Furthermore, the system utilizes a cloud-based backend, which supports scalability and ensures that the platform can handle an increasing number of users as the game grows.

3.7.7. Maintainability

The OmniRaid system is designed with maintainability as a key consideration to ensure that it can be easily updated, improved, and managed over time. The game follows a modular architecture approach, where different components such as gameplay systems, networking, AI modules, and user interface are developed independently. This allows developers to update or replace specific parts of the system without impacting the overall functionality.

To support efficient maintenance, the project adheres to clear coding standards and structured development practices, especially within the **Unity** environment using C#. Proper documentation is maintained throughout the development process, making it easier to understand system behavior, debug issues, and implement new features. Additionally, version control practices are followed to track changes, manage updates, and ensure stability. This approach ensures that OmniRaid remains scalable, adaptable, and easier to maintain in the long term.

Chapter 4

DESIGN AND ARCHITECTURE

4. Design And Architecture

The Design and Architecture chapter of the *OmniRaid* system explains how the game is structured, how its core components interact with each other, and how data flows throughout the system during gameplay. This chapter provides a clear overview of the system's architecture, including the game client, networking layer, and backend services, along with the internal data representation, process flows, and design models used to build the system.

It highlights the technical decisions made to ensure high performance, low latency, security, scalability, and maintainability in a real-time multiplayer environment. The design integrates the game client developed in Unity, real-time networking handled by Photon Fusion, and backend services managed through PlayFab. By clearly presenting the system architecture and design approach, this chapter demonstrates how OmniRaid fulfills both its functional and non-functional requirements while delivering a smooth, responsive, and reliable multiplayer gaming experience.

4.1. System Architecture

The system architecture of *OmniRaid* defines the overall structure of the game and explains how its major components work together to deliver a smooth and real-time multiplayer experience. It provides a clear view of how the client-side game, networking system, and backend services interact with each other during gameplay. Each module in the system is designed to perform a specific function, such as handling player input, managing game logic, synchronizing states, or storing player data.

The architecture ensures that all components communicate efficiently while maintaining low latency, high performance, and data consistency. By organizing the system into well-defined layers, OmniRaid achieves seamless interaction between gameplay systems and backend services, allowing players to experience responsive controls, synchronized multiplayer actions, and persistent progression without interruptions.

4.1.1. Architecture Style

OmniRaid is developed using a hybrid client-server architecture that combines real-time multiplayer networking with cloud-based backend services. The system follows a modular and component-based design approach inspired by object-oriented principles within the **Unity** environment.

The client layer consists of the game application where rendering, input handling, and local gameplay logic are executed. The server layer, powered by **Photon Fusion**, acts as an authoritative system that manages game state, validates player actions, and ensures fair gameplay through synchronization and lag compensation. The backend layer, implemented using **PlayFab**, is responsible for managing persistent data such as player profiles, leaderboards, inventory, and progression.

This separation of responsibilities improves maintainability, allows scalability as more players join the system, and ensures that gameplay remains secure and organized.

4.1.2. Components

The OmniRaid system is composed of several interconnected components, each designed to perform a specific role within the architecture. These components work together to ensure smooth

gameplay, real-time communication, and secure data management. By dividing the system into modular parts, the architecture becomes easier to maintain, extend, and optimize without affecting the entire system.

4.1.2.1.Game Client (User Interface and Gameplay System)

The game client serves as the primary interface through which players interact with OmniRaid. It is responsible for rendering the game environment, managing scenes, handling player input, and running the core gameplay loop. The client also performs client-side prediction to provide smooth and responsive controls, even before server confirmation is received.

Additionally, the user interface includes menus, HUD elements, and feedback systems that allow players to navigate the game, view statistics, and interact with different features. This component ensures that the overall user experience remains intuitive and engaging.

4.1.2.2.Networking System (Photon Fusion Server)

The networking system is responsible for enabling real-time multiplayer functionality. It is implemented using **Photon Fusion**, which provides an authoritative server model. This component handles state synchronization, lag compensation, and state reconciliation to ensure that all players see a consistent and fair version of the game world.

It processes input received from clients, validates actions, and then distributes the updated game state back to all connected players. This ensures accurate gameplay interactions, especially in fast-paced combat scenarios.

4.1.2.3.Backend System (Player Data Management)

The backend system is implemented using **PlayFab** and is responsible for managing all persistent data. This includes player profiles, inventory systems, leaderboards, and progression data. It ensures that player achievements, rankings, and customizations are securely stored and accessible across sessions.

The backend also supports cloud-based operations, allowing the system to scale efficiently and handle increasing numbers of players without performance degradation.

4.1.2.4. Matchmaking and Game Logic System

This component manages how players are grouped into matches based on factors such as skill level and network conditions. It ensures balanced gameplay by creating fair matches and minimizing latency differences between players.

In addition to matchmaking, this system also coordinates core game rules, session management, and interaction between different gameplay modules, ensuring a smooth multiplayer experience.

4.1.3. Interactions

In OmniRaid, all components communicate in a structured and efficient manner to maintain real-time gameplay. The game client sends player input data to the server, which processes and validates these actions using the networking system. The server then updates the game state and sends synchronized data back to all clients.

At the same time, the client interacts with the backend services to fetch and update player-related data such as profiles, rankings, and progression. For example, when a match ends, the system sends player performance data to the backend, which updates experience points and leaderboard rankings.

This continuous flow of data between the client, server, and backend ensures that gameplay remains responsive, synchronized, and consistent for all players.

4.1.4. System Architecture Diagram

The system architecture diagram illustrates the complete structure of the OmniRaid system and how its components interact. It shows the relationship between the Unity client, the Photon Fusion server, and the PlayFab backend. The diagram highlights how player input flows from the client to the server, how the server processes and synchronizes game state, and how backend services handle persistent data.

The diagram also represents internal components such as input handling, client prediction, state synchronization, lag compensation, and matchmaking logic. It clearly demonstrates how data flows between these components to support real-time gameplay and backend operations.

Overall, architecture ensures a clear separation of responsibilities, efficient communication between modules, and a scalable design that supports future enhancements and increased player load.

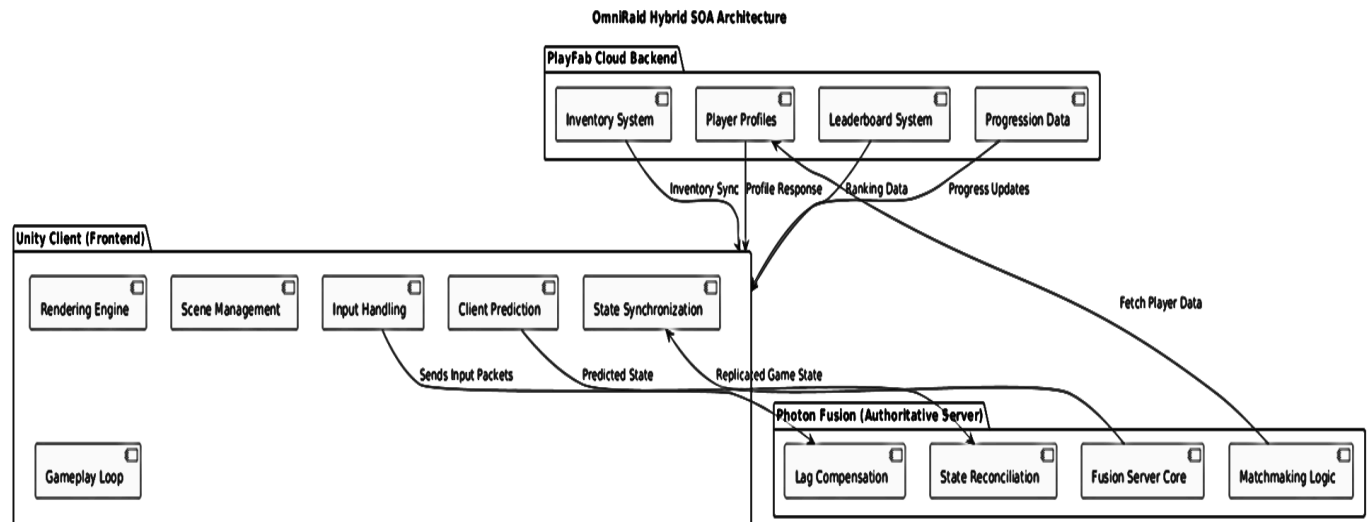


Figure 4.1.4.12: OMNIRAID Architecture Diagram

4.2. Data Representation

In the context of *OmniRaid*, data representation refers to how game-related information is structured, organized, and managed throughout the system. It defines how different types of data such as player profiles, match records, inventory items, and leaderboard rankings are formatted and handled within the game. Since OmniRaid is a real-time multiplayer system, data representation plays an important role in ensuring that information is processed efficiently and consistently across all components.

The system represents data in a structured format so that it can be easily transmitted between the client, server, and backend services. For example, player-related data includes attributes such as player ID, username, experience points (XP), level, and rank, while match data includes details such as match ID, player statistics, and results. By organizing data in a consistent and logical way, the system ensures smooth communication, accurate synchronization, and reliable storage of information across sessions.

4.2.1. Data storage approach

4.2.1.1.PlayerProfile (PlayFab player data):

```
1 PlayerProfile.json > ...
2 {
3   "PlayFabId": "12345",
4   "Username": "PlayerOne",
5   "XP": 3250,
6   "Level": 12,
7   "Rank": "Gold II",
8   "Achievements": ["FirstBlood", "Sharpshooter"],
9   "Loadout": {
10    "primary": "AK47_v2",
11    "secondary": "Pistol_9mm",
12    "grenade": "Frag"
13  },
14  "Inventory": [
15    {"itemID": "skin_rifle_blue", "owned": true},
16    {"itemID": "skin_helmet_red", "owned": true}
17  ],
18  "MatchHistorySummary": {
19    "matchesPlayed": 120,
20    "wins": 55,
21    "kills": 1100
22  },
23  "LastOnline": "2025-11-28T10:34:00Z"
24 }
```

Figure 4.2.1.1.13: PlayerProfile Data

4.2.1.2.LeaderboardEntry:

```
1 LeaderboardEntry.json > ...
2 {
3   "PlayFabId": "12345",
4   "Username": "PlayerOne",
5   "Score": 4200,
6   "RankPosition": 123
7 }
```

Figure 4.2.1.2.14: LeaderBoard Data in PlayFab

4.2.1.3.MatchRecord:

```
{ } MatchRecord.json > ...
1  {
2    "matchID": "m-20251128-0001",
3    "players": [{ "PlayFabId": "12345", "kills": 10, "deaths": 4 }],
4    "winnerTeam": "Blue",
5    "durationSeconds": 780,
6    "map": "Urban_Arena_01"
7  }
8  |
```

Figure 4.2.1.3.15: MatchRecord Data

4.2.2. Data Dictionary

4.2.2.1.Achievement (string)

In OmniRaid, an *Achievement* represents a specific in-game milestone or accomplishment unlocked by a player. These are stored as string identifiers (e.g., "First_Blood", "Headshot_Master", "Survivor_10_Matches").

From an SOA perspective, achievements are managed as a **separate service module** (Achievement Service). When a player completes a condition in a Photon Fusion match (server-authoritative), the server triggers an event that updates PlayFab data and notifies the Unity client.

This allows:

- Cross-session persistence
- Reward tracking (XP, cosmetics)
- Player engagement systems

4.2.2.2.GameSession (object)

A GameSession is a **server-hosted authoritative match instance** managed through Photon Fusion in OmniRaid.

It contains:

- List of active players (PlayerIDs)
- Map or level metadata
- Game mode configuration (e.g., Deathmatch, Capture the Flag)
- Match state (start time, score, timers, zone data)

In SOA terms, this belongs to the **Matchmaking + Session Management Service**, where sessions are dynamically created, managed, and destroyed.

Unity clients only receive a **synced snapshot**, while Photon Fusion ensures real-time consistency and lag compensation.

4.2.2.3.Inventory (array)

Inventory represents the collection of all items owned by a player in OmniRaid, including:

- Cosmetic skins (weapons, characters)
- Functional items (boosters, loadout unlocks)

It is stored as an **array of item objects**, each containing item ID, type, rarity, and ownership metadata.

In SOA architecture:

- Managed by an **Inventory Service (PlayFab-based backend)**
- Synced to Unity client at login and after match rewards
- Updated after in-game purchases or unlocks

4.2.2.4.Loadout (object)

Loadout defines the **active equipment configuration** a player uses in a match.

It includes:

- Primary weapon
- Secondary weapon
- Attachments (scopes, grips, etc.)
- Grenades or utility items
- Perks or abilities

In OmniRaid, Loadout is critical because it is:

- Sent to Photon Fusion server before match start
- Validated server-side to prevent cheating
- Locked during competitive gameplay for balance

This ensures fairness in multiplayer sessions.

4.2.2.5.MatchHistory (object)

MatchHistory is a structured record of all past matches played by a user.

It includes:

- Kills / deaths / assists
- Win/loss result
- Duration of match
- XP gained
- Ranking changes (if any)

This data is aggregated through a **Telemetry + Analytics Service** in OmniRaid, often stored in PlayFab or a dedicated analytics backend.

It is used for:

- Performance tracking
- Skill-based matchmaking (SBMM)
- Player progression insights

4.2.2.6.PlayerID (PlayFabId) (string)

PlayerID is a globally unique identifier assigned by PlayFab when a user registers or logs into OmniRaid.

It is the **primary key for all player-related data**, linking:

- Inventory
- XP progression
- Match history
- Achievements

In SOA design, this ensures **service decoupling**, meaning every microservice references the same identity without storing sensitive authentication data locally.

4.2.2.7.XP (integer)

XP (Experience Points) represent the player's progression metric in OmniRaid.

Players earn XP through:

- Killing enemies
- Winning matches
- Completing objectives
- Unlocking achievements

XP is processed server-side to prevent manipulation and is stored centrally in PlayFab. It acts as the raw input for level calculation.

4.2.2.8.Level (integer)

Level is a **derived progression value calculated from XP thresholds**.

For example:

- Level 1: 0–1000 XP
- Level 2: 1001–2500 XP
- Level 3: 2501–5000 XP

In OmniRaid, Level is not directly edited—it is always computed from XP using a progression formula.

It impacts:

- Matchmaking tiers
- Unlockable content
- Ranked system eligibility

4.3. Process Flow

4.3.1. Gameplay Flow:

The Gameplay Activity Diagram summarizes the complete player session flow from main menu to post-match progression saving. It shows key steps including mode selection, matchmaking, lobby entry, gameplay loop, match completion, reward calculation, and saving XP to PlayFab. This diagram outlines how major subsystems coordinate to deliver a smooth multiplayer experience.

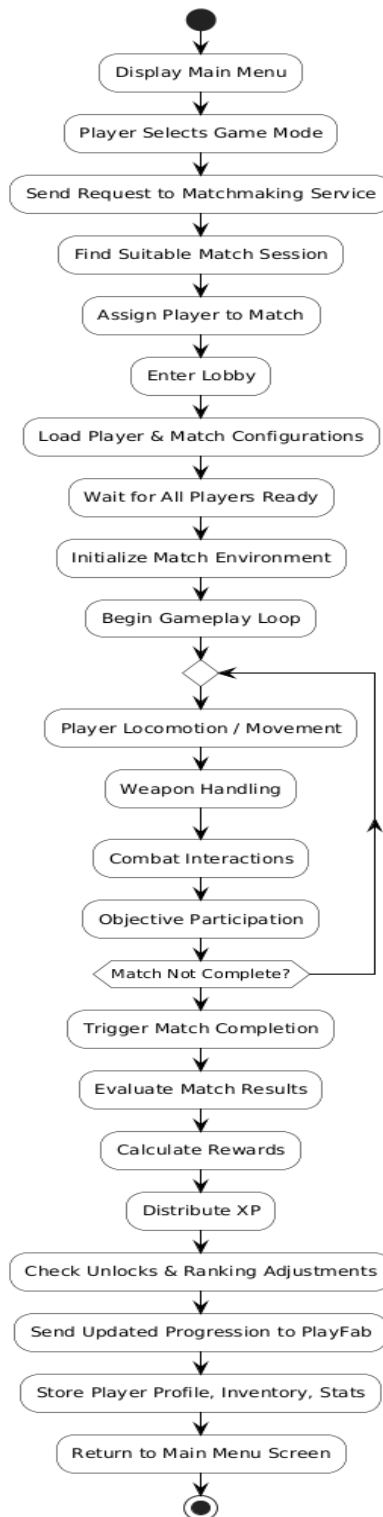


Figure 4.3.1.16: Activity Diagram of Gameplay System

4.3.2. Shooting Flow:

The Shooting Activity Diagram explains the firing process from player input to server-validated hit results. The client triggers fire input and visual feedback, while the Photon Fusion server performs authoritative hit checks using lag compensation. Server-approved results are replicated to all clients, updating health, kill feed, and HUD elements. The diagram captures the core combat interaction cycle.

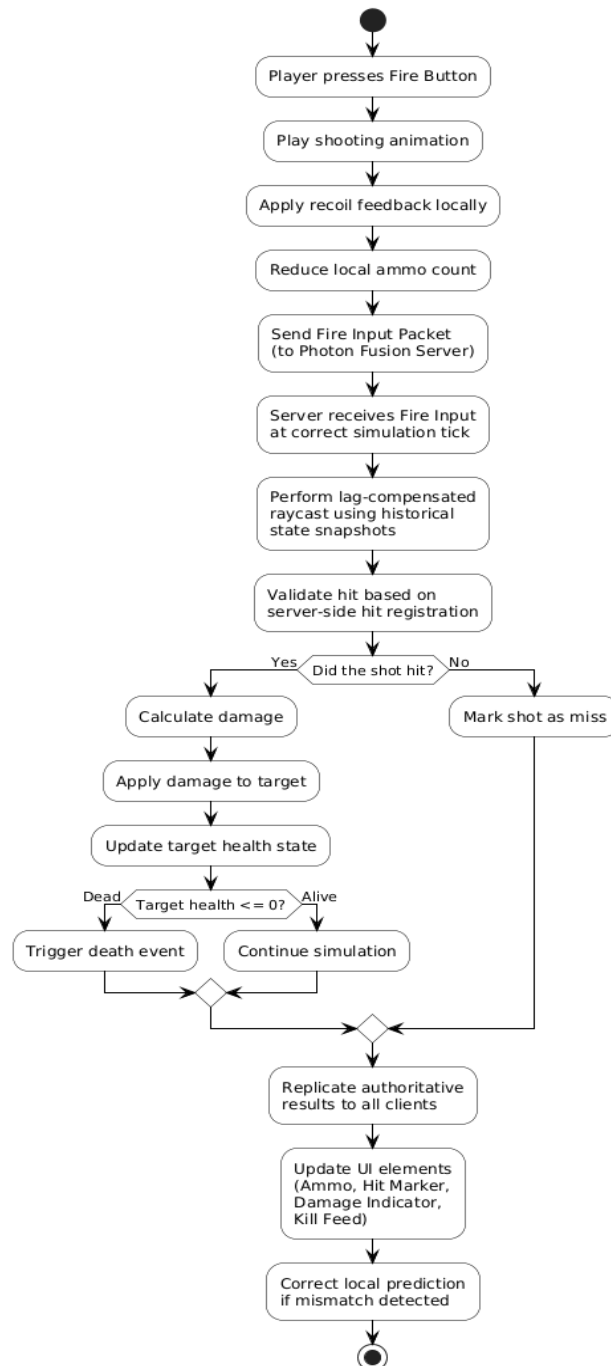


Figure 4.3.2.17: Activity Diagram of Shooting System

4.4. Design Models

4.4.1. Class Diagram

The UIManager interacts with the GameManager to update the HUD and display match results. The GameManager manages players, match state, and scoring, while the NetworkManager handles player inputs and network events.

Player data is saved and loaded through the PlayFabManager, which also updates leaderboards and achievements. The ProgressionManager tracks experience points and unlockable features for each player. Players and AI Bots both interact with Weapon objects for combat functionality. The AI_Bot evaluates threats, navigates, and uses weapons, while the Player performs actions like moving, attacking, and reloading.

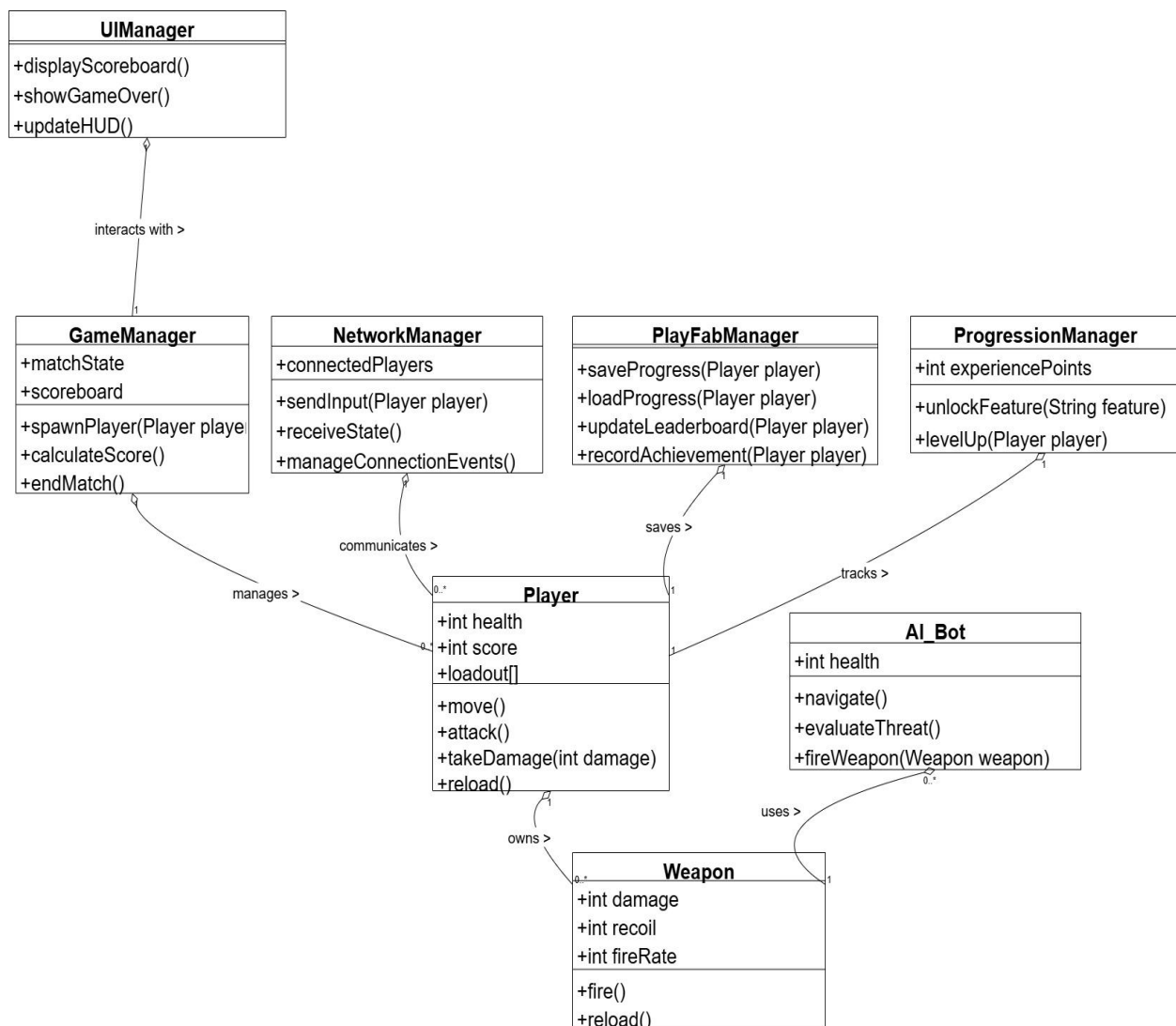


Figure 4.4.1.18: Class Diagram of OmniRaid

4.4.2. Sequence Diagrams

Sequence Diagram A — AI Bot Enemy Detection

This diagram shows how an AI Bot detects and engages a player. The bot scans the environment, selects a target, requests a navigation path, and decides to attack. It then triggers a firing event validated by the Fusion server, which synchronizes the final result to clients. It illustrates the decision-making and combat sequence of AI behavior.

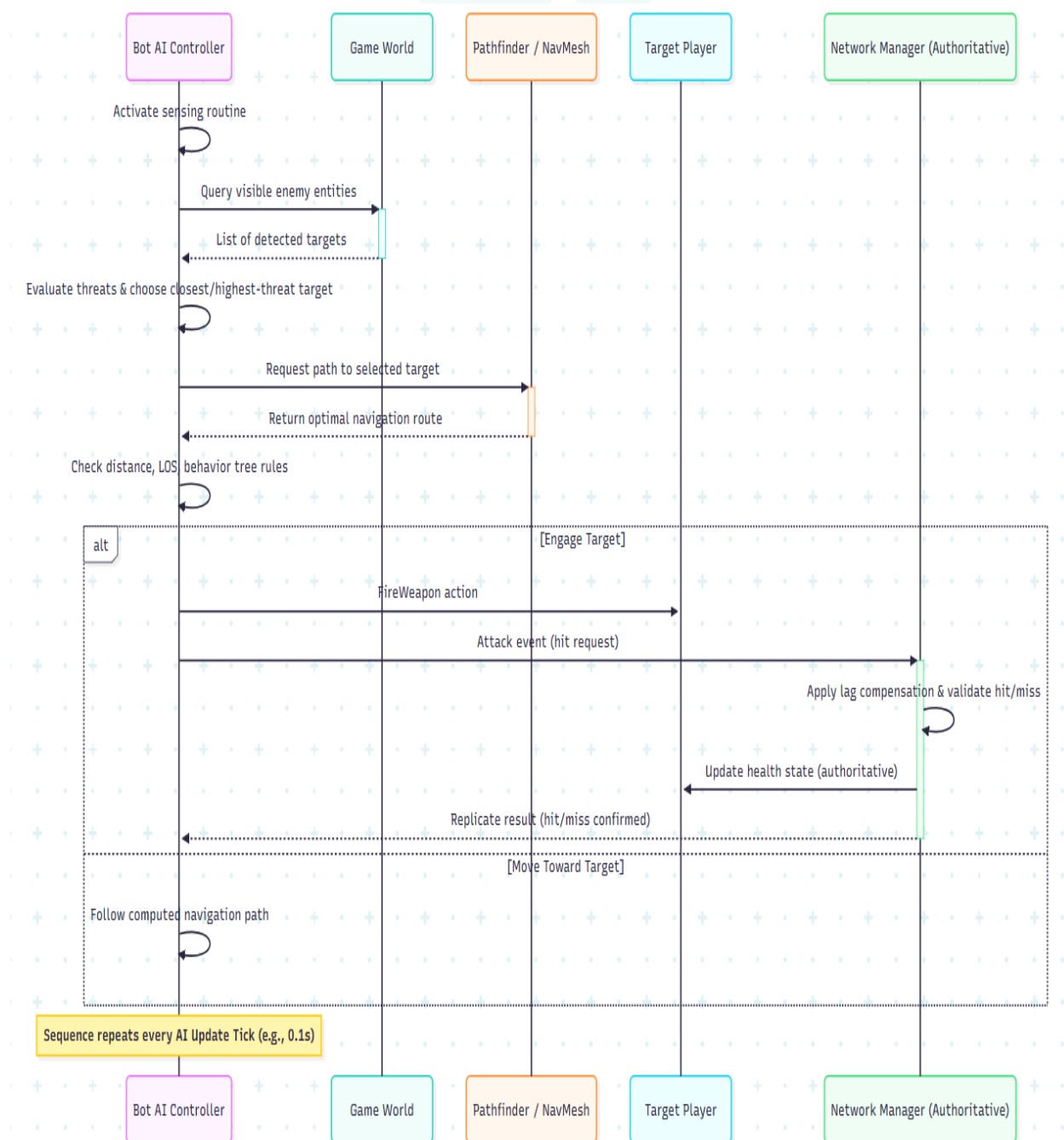


Figure 4.4.2.19: Sequence Diagram of Omniraid

Sequence Diagram B — Player Progression Update

The diagram outlines how player progression is saved after a match. The server computes match stats, sends results to the client, and updates XP and rewards through PlayFab using JSON API calls. PlayFab confirms the save, and the client updates the UI. It represents the workflow of persistent player progression.

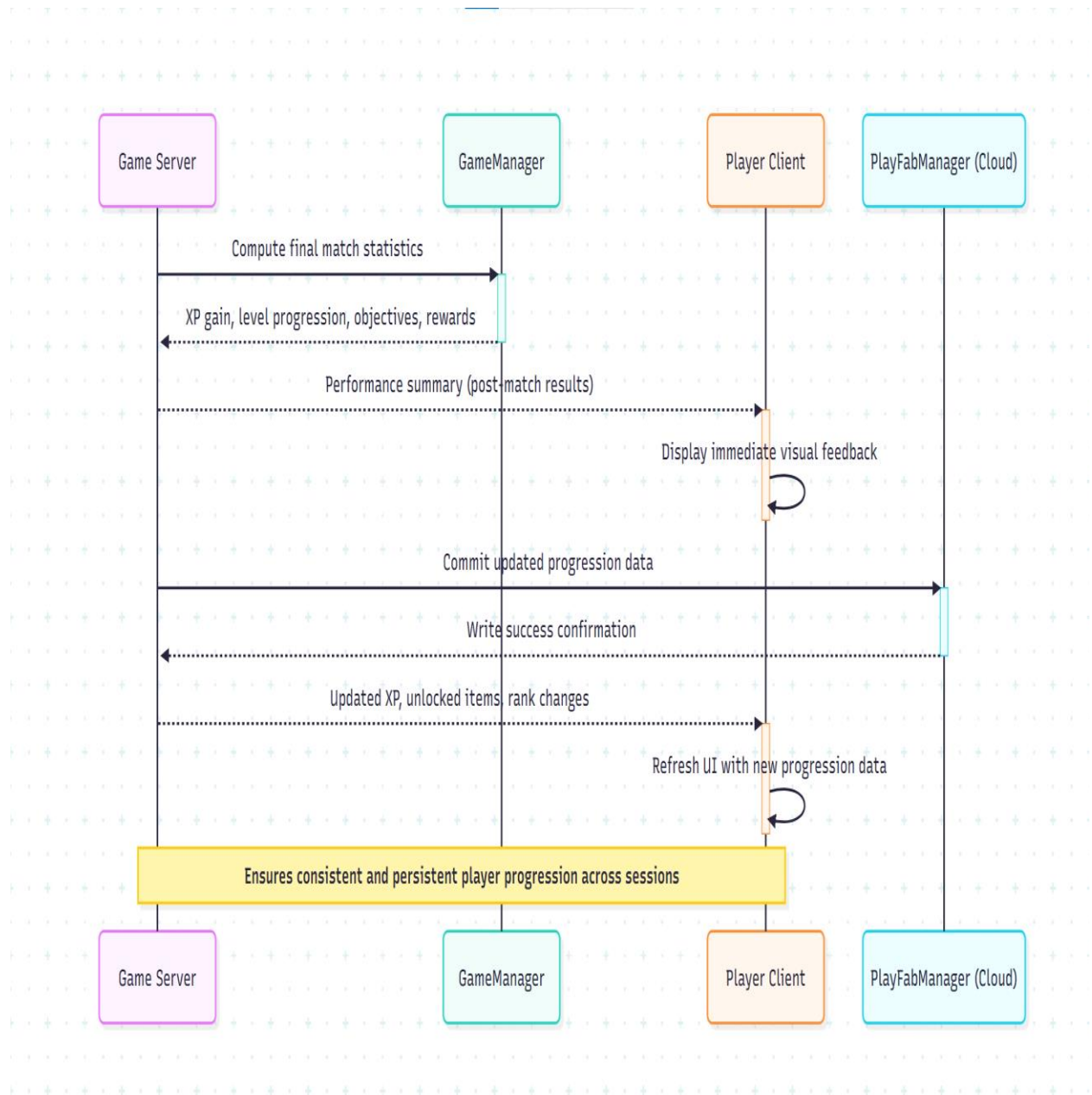


Figure 4.4.2.20: Sequence Diagram of Omniraid

Chapter 5

IMPLEMENTATION

5. Implementation

This chapter explains how the OmniRaid system was implemented at the technical level. It does not include source code but provides detailed descriptions of the core algorithms, networking mechanisms, external services and user interface design decisions used to build the game. The purpose of this chapter is to demonstrate how the proposed architecture was transformed into a fully functional multiplayer system, ensuring real-time performance, low latency, synchronization accuracy, and secure data handling.

5.1. Algorithms

This section describes the core algorithms used in the OmniRaid system. Each algorithm defines the internal logic behind key gameplay and system functionalities such as AI behavior, matchmaking, combat validation, and player progression. These algorithms are implemented using a combination of Unity, Photon Fusion's authoritative networking model, and backend services like PlayFab. The descriptions are presented in structured steps and pseudocode to clearly illustrate how different modules operate within the game environment.

5.1.1. AI Pathfinding Algorithm

The AI Pathfinding algorithm in OmniRaid enables bots to navigate the game environment efficiently while adapting to dynamic combat situations. It ensures that non-player characters (NPCs) can move intelligently across the map, whether they are patrolling or chasing enemies. The system primarily relies on Unity's NavMesh for optimal path calculation, with a fallback mechanism using predefined waypoints when NavMesh data is unavailable or incomplete. This approach guarantees reliable navigation even in complex or partially loaded environments.

Steps:

1. The system receives the current position of the bot (agent) and the target position.
2. It checks whether a valid NavMesh path exists between the two positions.
3. If a NavMesh path is available:
 - a. Calculate the optimal path using NavMesh pathfinding.
 - b. Return the computed path for navigation.
4. If a NavMesh path is not available:
 - a. Use a fallback mechanism based on predefined waypoints.
 - b. Generate an alternative path using waypoint connections.
 - c. Return the fallback path.
5. The bot follows the generated path to reach the target position.

5.1.2. AI Decision Algorithm

The AI Decision algorithm controls the behavior of bots in OmniRaid by allowing them to dynamically respond to the game environment. It enables bots to either patrol predefined routes or engage enemies based on real-time conditions. This algorithm enhances gameplay realism by simulating intelligent decision-making similar to human players.

Steps:

1. The bot scans the environment to detect the nearest enemy target.
2. If no enemy is detected:
 - a. Retrieve the predefined patrol route assigned to the bot.
 - b. Move the bot toward the next patrol point.
3. If an enemy is detected:
 - a. Calculate the distance between the bot and the target.
 - b. If the target is within weapon range:
 - i. Engage the target by initiating attack behavior.
 - c. If the target is outside weapon range:
 - i. Compute a path toward the target using the pathfinding algorithm.
 - ii. Move the bot along the computed path.
4. Continuously update decisions based on changes in enemy position and environment.

5.1.3. Hit Registration & Lag Compensation Algorithm

This algorithm ensures accurate and fair hit detection in OmniRaid's multiplayer environment by using Photon Fusion's authoritative server model. It compensates for network latency by validating player actions on the server rather than relying solely on client input. This prevents cheating and ensures consistency across all players.

Steps:

1. The client sends a fire event containing input data (e.g., aim direction, timestamp).
2. The server retrieves the current authoritative game state.
3. The system performs hit validation using lag compensation techniques:
 - a. Reconstruct past player positions based on tick rate and latency.
 - b. Compare the client's input with the reconstructed server state.
4. If a valid hit is detected:
 - a. Apply damage to the target player.
 - b. Update the authoritative game state.
 - c. Replicate the updated state to all connected clients.
5. If no valid hit is detected:
 - a. Treat the event as a miss.
 - b. Replicate the miss event to clients for visual feedback.

5.1.4. Matchmaking Algorithm

The Matchmaking algorithm in OmniRaid is responsible for grouping players into balanced matches based on skill level and network conditions. It ensures fair gameplay by considering

factors such as MMR (Matchmaking Rating) and latency (ping). This system improves player experience by reducing mismatches and connection issues.

Steps:

1. Add players to the matchmaking queue when they request a match.
2. Sort the queue based on player MMR and network latency.
3. For each player in the queue:
 - a. Identify a pool of candidate players within acceptable MMR and ping thresholds.
 - b. Check if the candidate pool meets the required number of players for a match.
4. If sufficient players are found:
 - a. Create a new match session using Photon Fusion.
 - b. Assign players to the match.
 - c. Remove matched players from the queue.
5. If insufficient players are found:
 - a. Keep players in the queue until suitable candidates are available.

5.1.5. Progression (XP Calculation) Algorithm

The Progression algorithm calculates the experience points (XP) earned by players after each match based on their performance. It encourages player engagement by rewarding active participation, skill, and objective completion. The calculated XP is then used to update the player's level and unlock new content.

Steps:

1. Retrieve the player's match performance data, including time played, kills, assists, and objectives completed.
2. Calculate base XP based on total time played.
3. Calculate additional XP:
 - a. Kill XP based on number of kills.
 - b. Assist XP based on number of assists.
 - c. Objective XP based on completed objectives.
4. Sum all XP components to compute total XP.
5. Update the player's XP in the backend (PlayFab).
6. Check if XP exceeds level thresholds:
 - a. If yes, increase player level.
 - b. Unlock rewards if applicable.
7. Store updated progression data for future sessions.

5.2. External API's:

The OmniRaid system relies heavily on external APIs and cloud-based services to implement a scalable, real-time multiplayer architecture. Unlike traditional web-based systems, OmniRaid integrates **game networking APIs (Photon Fusion)** and **backend-as-a-service solutions (PlayFab)** to handle complex operations such as real-time synchronization, matchmaking, player authentication, and persistent data storage.

These APIs are tightly coupled with the Unity game engine and are used across both the client and server layers. The Unity client is responsible for capturing player input and rendering the game, while Photon Fusion ensures real-time communication and authoritative state management. PlayFab, on the other hand, acts as the persistent backend, storing all long-term player data such as inventory, achievements, XP, and match history.

The interaction flow typically works as follows: when a player performs an action (e.g., joining a match, updating loadout, or completing a game), the Unity client sends requests through Photon Fusion or PlayFab APIs. Photon Fusion processes real-time gameplay events and synchronizes them across all connected players, while PlayFab securely stores and retrieves persistent data. This separation of responsibilities ensures high performance during gameplay and reliable long-term data management.

Table 5.2.20: Details of APIs Used in OmniRaid

Name of API	Description of API	Purpose of Usage	Function/Class Name in which Used
Photon Fusion API	Real-time multiplayer networking framework with authoritative server model	Handle game sessions, state synchronization, lag compensation, and player interactions	GameSessionManager, NetworkRunner (Unity Client & Server)
PlayFab API	Cloud-based backend service for player data management	Manage player authentication, inventory, XP, level progression, and cloud storage	PlayFabManager, AuthService, InventoryService
PlayFab Authentication API	Provides secure login and user identity management	Authenticate players using PlayFab ID and maintain secure sessions	AuthController (Unity Client + PlayFab SDK)
PlayFab Data API	Stores and retrieves player-related data	Save match history, XP, achievements, and player statistics	PlayerDataService (Backend Logic / Unity Integration)

PlayFab Inventory API	Manages player-owned items and loadouts	Handle cosmetic items, weapons, and inventory persistence	InventoryManager (Unity Client + PlayFab)
Photon Transport Layer	Handles low-level network communication	Ensure fast and reliable data transfer between clients and server	NetworkRunner/ Transport Layer (Photon Fusion)

5.3. User Interface

5.3.1. Main Menu

This screen shows the main lobby interface of the game OmniRaid. On the left, the player can navigate through options such as Play, Inventory, Shop, Settings, and Exit Game. The center area displays various match modes, including Team Deathmatch, Free-for-All, and Custom Match options. At the top-right, the player profile is visible, showing the username, level, gold balance, and premium currency. The background features a futuristic hangar with soldiers and a dropship, giving a military sci-fi atmosphere. A matchmaking panel at the bottom provides status updates and estimated wait time. A large Find Match button allows the player to start matchmaking in the selected game mode.

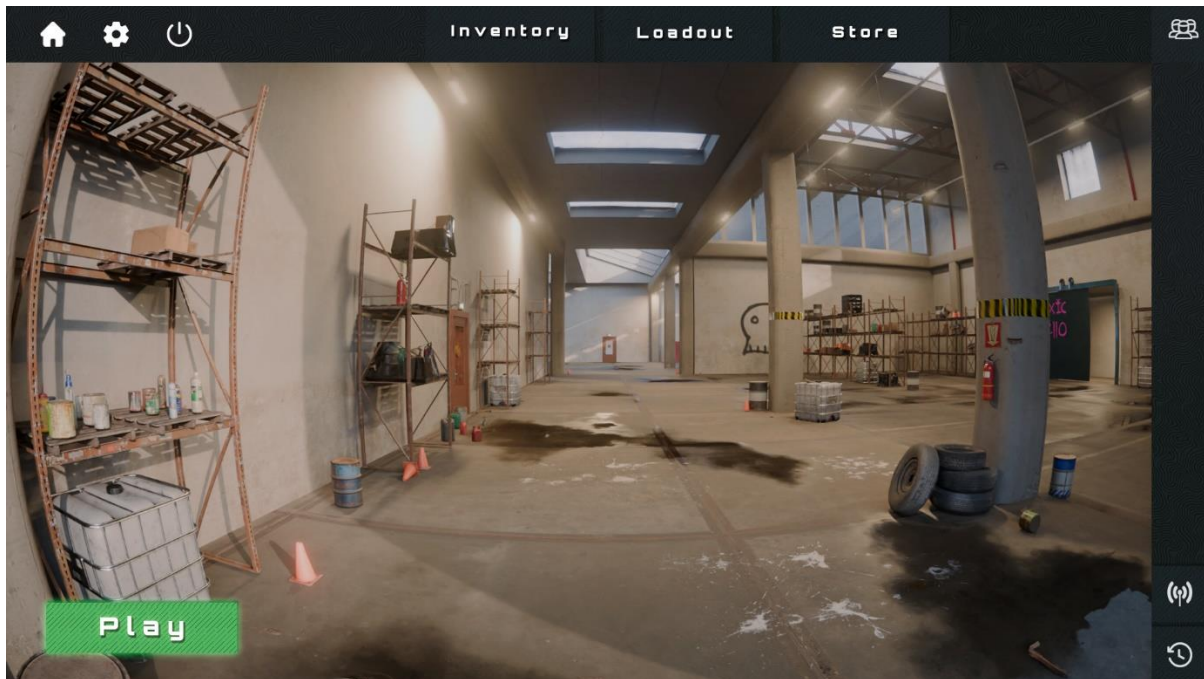


Figure 5.3.1.21: Main-Menu

5.3.2. Lobby

This screen displays the Select Game Mode interface of the game, showing multiple matchmaking options. Each game mode is presented as a card with a map preview, description, team size, and estimated wait time. Available modes include Team Deathmatch, Battle Royale, Objective Control, and Squad. The selected mode (Team Deathmatch) is highlighted with a glowing border and a Confirm button. On the right side, Ranked Play is shown but locked, requiring a specific player level to access. The background shows a realistic military hangar with soldiers preparing for deployment. A status bar at the bottom indicates that matchmaking is in progress.

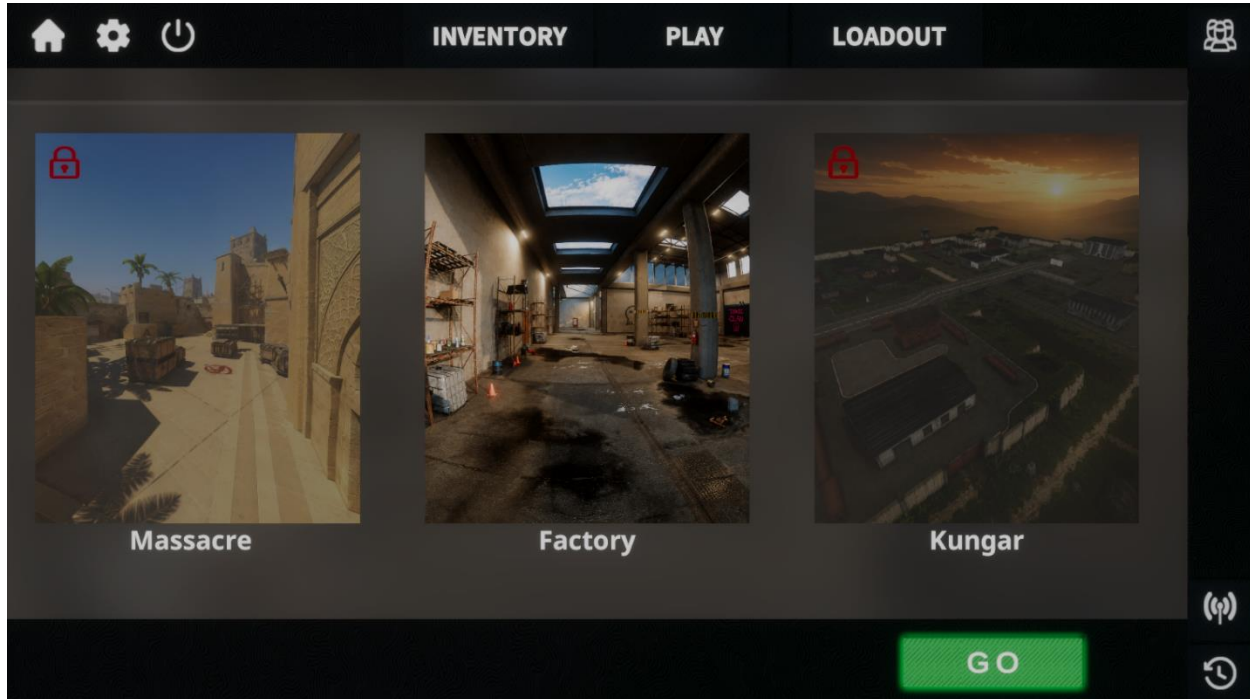


Figure 5.3.2.22: Looby Interface

5.3.3. Inventory

The Inventory screen displays the player's weapon collection in a structured grid layout categorized by weapon type. Tabs at the top allow switching between Primary Weapons, Secondary Weapons, Melee, Grenades, Character Skins, and Consumables. Each weapon card shows its rarity through color coding such as Rare, Epic, and Legendary. Locked items are visually marked with lock icons and unlock requirements. The selected weapon appears on the right side in a large 3D preview. Detailed weapon statistics including damage, fire rate, accuracy, and recoil are displayed beside the preview.

The currently equipped weapon is clearly labeled for quick identification. Player currency (gold and credits) is shown at the top corner for easy access. Action buttons such as "Equip to Loadout," "Save Changes," and "Cancel" allow quick decision making. A confirmation message indicates that loadout changes are synced with PlayFab. The overall layout is clean, minimal, and inspired by modern FPS inventory systems like CS2.

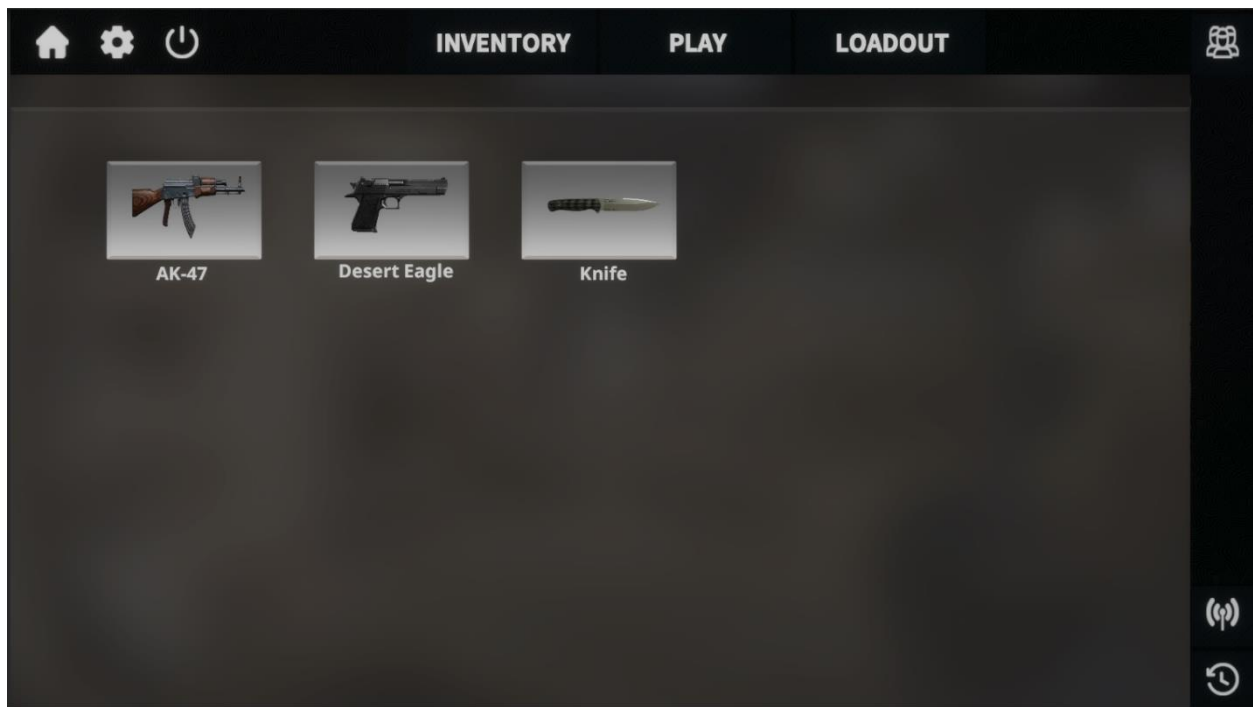


Figure 5.3.3.23: Player Inventory

5.3.4. Shop

The Shop screen is organized into multiple tabs including Featured, Skins, Crates, and Currency, allowing players to easily browse different categories of purchasable items. The player's profile icon, along with their available gold and credits, is displayed in the top-right corner for quick reference. The Featured tab highlights limited-time offers such as the Weekly Bundle, which includes weapon skins and gold packs with clearly visible prices. A Rare Crate x5 offer is displayed in the main panel, showcasing an Epic Plasma Rifle Skin as a possible reward alongside multiple loot crates.

A red warning message appears indicating "Insufficient Funds," clearly showing the required gold and the player's current balance. The Apply button remains disabled until enough currency is available. Additional perk items such as XP boosts and consumables are displayed at the bottom with duration and quantity labels. The interface uses glowing borders, rarity-based color coding, and bold button labels to enhance visibility. Real-time currency feedback prevents accidental purchases and maintains clarity. Overall, the shop follows a clean, futuristic FPS design style that encourages browsing, upgrades, and player progression.

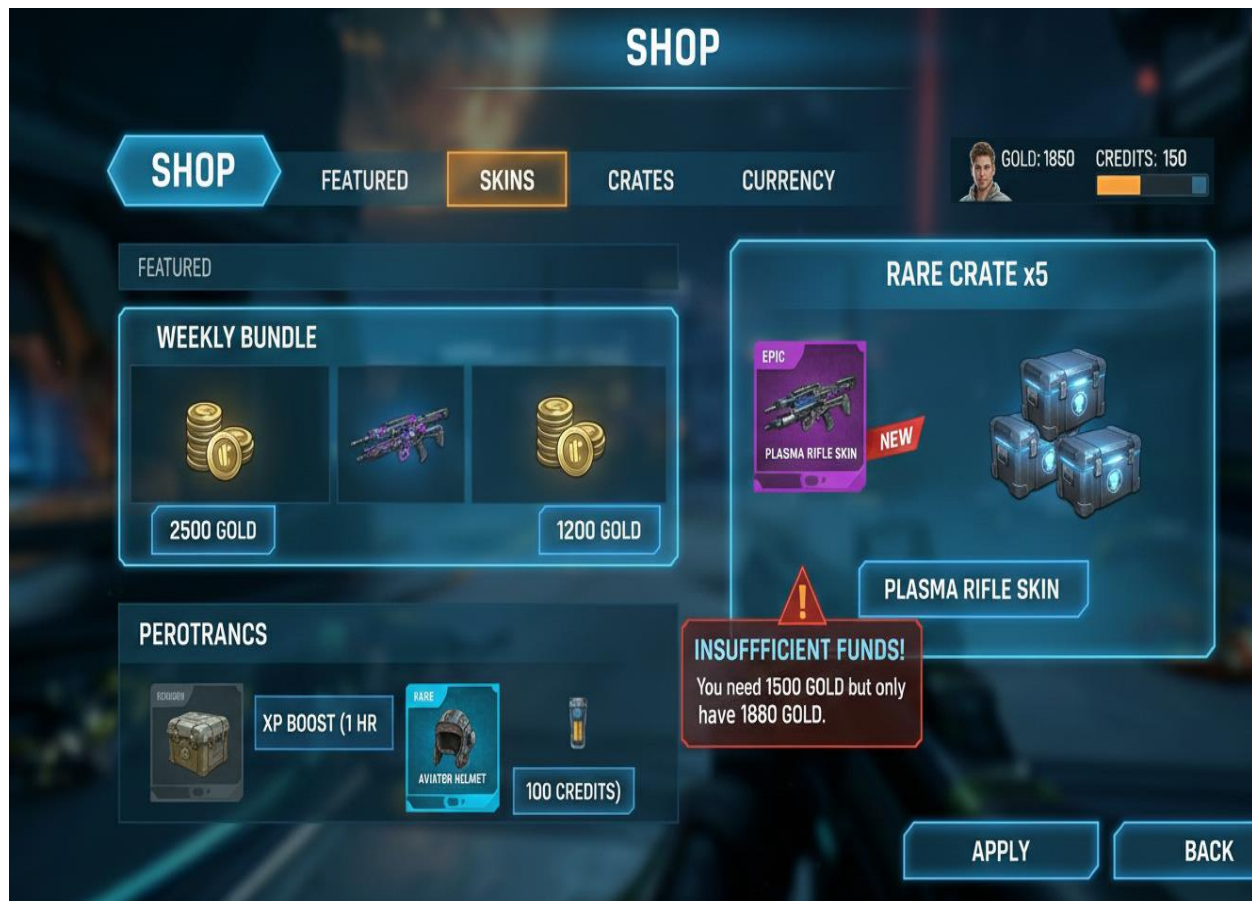


Figure 5.3.4.24: InGame Shop UI

5.3.5. Containers

At the center of the screen, a glowing container is displayed with a prompt to select and open a crate. The background remains slightly blurred to keep the focus on the reward system. Below the container, multiple reward cards are shown representing possible items obtained from the crate. These include cosmetic skins, helmets, supply crates, and in-game currency. Each reward card is highlighted with a distinct rarity color such as Epic, Rare, and Common. The Epic Plasma Rifle Skin is showcased prominently with a purple glow to indicate its high value. The Rare Aviator Helmet is displayed with a blue rarity outline.

A Common Supply Crate is shown in a faded state to represent a lower-tier reward. A green currency card displays the reward of 150 credits. The “Open Crate” button is placed below the container for user interaction. A real-time system notification confirms that the selected items have been added to the inventory through PlayFab synchronization. The confirmation message is displayed in a clean status bar at the bottom of the screen. The “Claim All” button allows the player to collect all earned rewards instantly. A “Back” button is provided to return to the previous menu. Subtle glow and particle effects enhance the reward reveal experience. Visual feedback reinforces item rarity and progression. The layout ensures that rewards are clearly visible and easily

understandable. The interface maintains a futuristic FPS-style design theme. The reward system encourages player engagement and progression through visual satisfaction.

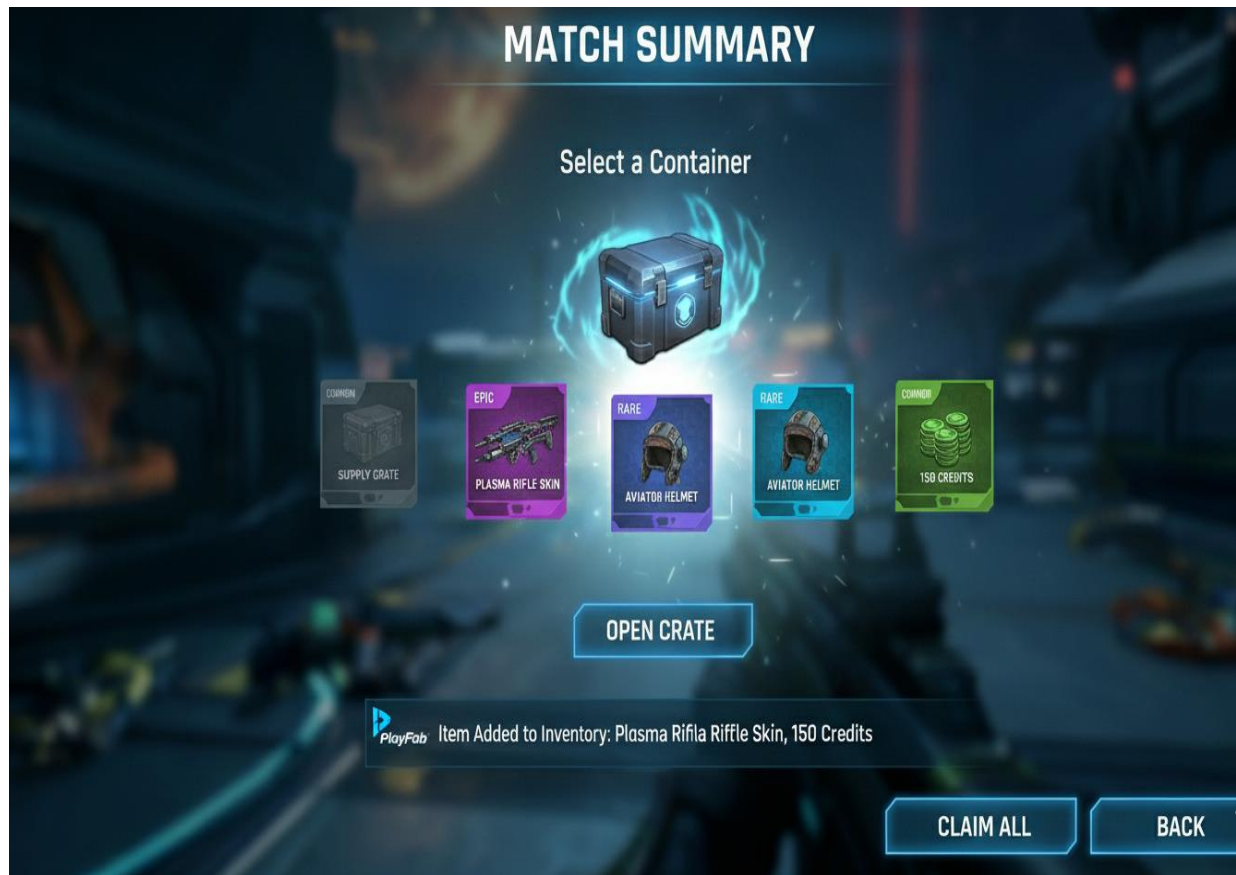


Figure 5.3.5.25: Opening Interface

5.3.6. Settings

The image shows a futuristic game settings menu displayed against a high-tech cityscape background with tall glowing skyscrapers. At the top, a horizontal navigation bar presents different tabs: Gameplay, Controls, Video, Audio, and Network, with the Gameplay tab currently selected. The left section features a Sensitivity panel showing mouse sensitivity set to 1.2 and controller sensitivity set to 4, each adjustable through sliders. Below that, a Controls section offers buttons for Keybinds and Gamepad Layout configuration. Further down on the left is a Graphics section with options for resolution set to 1920×1080, quality set to high, and V-Sync toggled on.

On the right side, another Graphics panel mirrors similar settings with matching resolution and quality options. Beneath the graphics settings, an Audio section displays sliders for master volume, music, and SFX levels with a visual sound graph. Lower on the right is a Network section showing the latency at 52 ms along with a toggle to show latency. At the bottom of the menu, two buttons—Apply and Cancel—allow the user to confirm or discard changes. The design uses a sleek blue-gray theme that matches the sci-fi environment. The interface appears polished, minimalistic, and gamer-friendly. Soft lighting and blurred buildings in the background enhance depth. The settings window floats semi-transparently above the scene. Icons above each tab add clarity to navigation. The entire layout is structured, intuitive, and visually immersive.



Figure 5.3.6.26: Setting Interface

5.3.7. Match Result Screen

The interface is divided into sections highlighting the player's performance, progression, and earned rewards. At the top left, the Performance panel shows kills marked at 24, assists at 8, and deaths at 12, each represented with distinct icons. Below it, a Progression bar tracks the player's XP journey, revealing that 3500 XP has been earned during the match. The bar stretches from Level 45 toward Level 46, nearly reaching the next milestone. Beneath the XP bar, a message highlights new unlocks: a plasma rifle skin and an aviator helmet, giving the player cosmetic customization options. On the right side, a Rewards panel presents two items: 500 credits and a rare crate, both visually framed with glowing effects.

A blue button labeled "Claim Rewards" invites the player to collect them. At the bottom center, the match result shows the Allied Team scoring 150 points while the Enemy Team scored 125, indicating a victory. A note confirms that PlayFab has saved the stats successfully. Below the summary, two buttons allow the player to either claim rewards or exit to the main menu. The background features a blurred combat environment with neon lighting, emphasizing the game's

futuristic aesthetic. The UI is crisp, modern, and designed with glowing blue accents. It conveys clarity and satisfaction after gameplay.



Figure 5.3.7.27: Match Result Screen

Chapter 6

Testing and Evaluation

6. Testing and Evaluation

The Testing [11] and Evaluation phase of the OmniRaid system is a critical stage in the development lifecycle where the complete system is validated to ensure correctness, reliability, performance, and real-time responsiveness. Since OmniRaid is a multiplayer FPS game built using Unity, Photon Fusion, and PlayFab, testing is not limited to simple functionality checks but also includes real-time networking behavior, synchronization accuracy, server authority validation, and player progression consistency.

This phase ensures that every subsystem—such as gameplay mechanics, AI behavior, matchmaking, hit registration, and backend data persistence—works correctly both individually and when integrated with other modules. The goal of this stage is to confirm that the system performs as expected under real-world multiplayer conditions, including high latency, concurrent users, and dynamic gameplay scenarios.

Testing in OmniRaid is carried out using both manual testing and automated testing approaches. Manual testing helps simulate real player behavior in gameplay scenarios, while automated testing ensures repeated validation of critical systems such as XP calculation, matchmaking logic, and server-side validation.

6.1. Manual Testing

6.1.1. System Testing

System testing in OmniRaid is performed after the full implementation of core modules such as networking, AI bots, matchmaking, and player progression systems. The primary objective is to evaluate the system as a complete and integrated multiplayer environment rather than isolated components.

This testing [11] phase ensures that Unity client, Photon Fusion server, and PlayFab backend work together seamlessly, maintaining synchronization, fairness, and persistence of game data.

6.1.1.1. Purpose of System Testing

The purpose of system testing in OmniRaid is to ensure that the entire multiplayer architecture operates correctly under real gameplay conditions. It validates that:

- The game sessions are created and managed correctly through Photon Fusion
- Player actions such as movement, shooting, and interactions are synchronized across all clients
- Backend systems like PlayFab correctly store and retrieve player data
- Match results, XP, and achievements are properly updated after each session
- The system remains stable under multiple concurrent players

System testing also helps identify issues such as latency spikes, desynchronization, incorrect hit registration, and matchmaking imbalance before deployment.

6.1.1.2. Types of System Testing

System testing in OmniRaid is divided into multiple categories to ensure complete validation of the system:

- **Unit Testing:**
Focuses on individual modules such as AI pathfinding, XP calculation, and authentication logic. Each function is tested independently to ensure correctness.
- **Functional Testing:**
Ensures that gameplay features such as login, matchmaking, combat system, and loadout selection behave according to the game design requirements.
- **Integration Testing:**
Verifies that different modules work together correctly, such as Unity client communication with Photon Fusion server and PlayFab backend synchronization for player data.
- **Network Testing (Multiplayer Specific):**
Evaluates real-time synchronization, lag compensation, and state reconciliation across multiple connected players.

6.1.1.3. Importance of System Testing

System testing is extremely important in OmniRaid due to its real-time multiplayer nature. It ensures that:

- The gameplay experience remains smooth and responsive
- Server-authoritative logic prevents cheating and inconsistencies
- Player progression data remains accurate and persistent
- AI bots and matchmaking systems behave as expected under load
- The system is stable even with high network traffic and multiple concurrent sessions

This stage guarantees that OmniRaid is reliable, scalable, and ready for deployment in a competitive multiplayer environment.

6.1.2. Unit Testing

Unit testing in OmniRaid focuses on verifying individual gameplay modules in isolation. Each unit represents a specific function or system such as login authentication, AI decision-making, hit registration, matchmaking, or XP calculation. The goal is to ensure that each module performs correctly before integrating it into the full multiplayer system.

Unit tests are executed by providing predefined inputs to a module and comparing the actual output with the expected result. Any inconsistencies are fixed before moving to integration testing.

Table 6.1.2.21: Unit Testing for OmniRaid System

Test ID	Module Name	Test Case / Test Script	Description	Input Data	Expected Result	Actual Result	Status
UT-01	Authentication System	Verify player login using valid credentials	Ensures player can successfully log into the game using PlayFab authentication	PlayerID: valid, Password: correct	Player successfully authenticated and redirected to main menu	As expected	Pass
UT-02	Authentication System	Verify login with invalid credentials	Checks system response for incorrect login attempts	PlayerID: valid, Password: wrong	Login rejected and error message displayed	As expected	Pass
UT-03	AI Pathfinding Module	Compute navigation path between agent and target	Ensures bot correctly calculates path using NavMesh or fallback system	Agent Position, Target Position	Valid path generated and returned	As expected	Pass
UT-04	AI Decision System	Bot behavior when enemy is detected	Tests decision-making logic for engagement or patrol	Enemy detected within range	Bot engages or follows path toward target	As expected	Pass

UT-05	Hit Registration System	Validate server-side hit detection	Ensures correct hit validation using authoritative server logic	Fire event + input target position	Damage applied only on valid hits	As expected	Pass
UT-06	Matchmaking System	Match players based on MMR and ping	Ensures fair grouping of players in a session	Player queue with MMR values	Balanced match created successfully	As expected	Pass
UT-07	XP Progression System	Calculate XP after match completion	Validates XP calculation based on performance metrics	Kills, assists, objectives, time played	Correct XP calculated and returned	As expected	Pass
UT-08	Inventory System	Load player items from PlayFab	Ensures inventory data is correctly retrieved	PlayerID	Inventory items loaded successfully	As expected	Pass

6.1.3. Functional Testing

Functional testing in OmniRaid focuses on verifying that all game features and functionalities operate according to the defined requirements and design specifications. Unlike unit testing, which checks individual modules in isolation, functional testing evaluates how each feature behaves from a user's perspective within the actual gameplay environment.

The main objective of this testing phase is to ensure that every core feature—such as login, matchmaking, gameplay mechanics, loadout selection, and progression system—produces the correct output when given specific inputs. It simulates real-world player interactions to confirm that the system responds accurately and consistently.

In OmniRaid, functional testing is particularly important because it validates:

- Whether players can successfully log in and access the game
- Whether matchmaking correctly places players into sessions
- Whether gameplay actions (movement, shooting, AI behavior) function as expected
- Whether match results, XP, and inventory updates are correctly processed

This ensures that the system not only works technically but also delivers a smooth and expected gameplay experience.

Table 6.1.3.22: Functional Testing for OmniRaid System

Test ID	Module Name	Test Case / Test Script	Description	Input Data	Expected Result	Actual Result	Status
FT-01	Authentication System	Login with valid credentials	Verify player can access the game using correct login details	PlayerID: valid, Password: correct	Player successfully logs in and main menu is displayed	As expected	Pass
FT-02	Authentication System	Login with invalid credentials	Ensure system rejects incorrect login attempts	PlayerID: valid, Password: incorrect	Error message displayed and access denied	As expected	Pass
FT-03	Matchmaking System	Join matchmaking queue	Verify player can enter matchmaking system	Player with valid MMR and ping	Player added to queue successfully	As expected	Pass
FT-04	Matchmaking System	Create match session	Ensure players are matched correctly into a session	Multiple players in queue	Balanced match created and GameSession initialized	As expected	Pass
FT-05	Gameplay System	Player movement synchronization	Verify movement is smooth and synchronized across clients	Movement input (WASD)	Character moves correctly and syncs in real-time	As expected	Pass
FT-06	Combat System	Shooting and hit detection	Validate firing mechanism and hit registration	Fire input + target in range	Valid hits register and damage is applied	As expected	Pass
FT-07	AI System	Bot decision-making behavior	Ensure bots respond correctly to player presence	Enemy detected within range	Bot engages or navigates toward player	As expected	Pass
FT-08	Loadout System	Apply selected loadout	Verify selected weapons and equipment are	Selected loadout configuration	Loadout correctly applied at match start	As expected	Pass

			applied in-game				
FT-09	Inventory System	Load player inventory	Ensure items are retrieved from backend correctly	PlayerID	Inventory items displayed correctly in UI	As expected	Pass
FT-10	Progression System	Calculate XP after match	Validate XP calculation based on performance	Match stats (kills, assists, objectives)	Correct XP calculated and displayed	As expected	Pass
FT-11	Progression System	Level up system	Ensure player level increases when XP threshold is reached	XP exceeds level threshold	Player level updated and rewards unlocked	As expected	Pass
FT-12	Data Persistence	Save match results to backend	Verify match data is stored in PlayFab	Match result data	Data successfully saved and retrievable	As expected	Pass

6.1.4. Integration Testing

Integration testing in OmniRaid focuses on verifying that different modules of the system work together correctly as a unified multiplayer environment. Since OmniRaid is built using a combination of Unity client, Photon Fusion networking, and PlayFab backend services, this phase is especially critical to ensure smooth communication and data flow between these components.

In this stage, individual modules that have already passed unit and functional testing are combined and tested as a group. The goal is to identify issues related to data exchange, synchronization, API communication, and workflow continuity. Integration testing ensures that actions performed in one module correctly affect other connected modules.

For example, when a player completes a match, the gameplay system (Photon Fusion) must correctly send results to the backend (PlayFab), which then updates XP and inventory. Any failure in this chain can break the user experience. Therefore, this testing phase ensures that end-to-end workflows function seamlessly.

Integration testing in OmniRaid validates:

- Unity client ↔ Photon Fusion server communication
- Photon Fusion ↔ PlayFab data synchronization
- Matchmaking → GameSession → Gameplay flow
- Match results → XP calculation → Data persistence

- Inventory and loadout data retrieval before gameplay

Table 6.1.4.23: Integration Testing for OmniRaid System

Test ID	Module Name	Test Case / Test Script	Description	Input Data	Expected Result	Actual Result	Status
IT-01	Authentication + Backend	Login and retrieve player data	Verify login integrates with PlayFab and retrieves player profile	Valid PlayerID and credentials	Player logs in and profile data (XP, inventory) is loaded	As expected	Pass
IT-02	Matchmaking + Networking	Matchmaking to GameSession creation	Ensure players are correctly transferred from queue to active session	Players with valid MMR and ping	GameSession created and players connected successfully	As expected	Pass
IT-03	Unity Client + Photon Fusion	Player movement synchronization across clients	Verify real-time synchronization of player actions	Movement inputs from multiple players	All clients see consistent and synchronized movement	As expected	Pass
IT-04	Combat + Server Validation	Fire event and hit registration	Ensure client input is validated by server and applied correctly	Fire input + target data	Server validates hit and applies damage consistently	As expected	Pass
IT-05	Gameplay + Backend	Match end and result processing	Verify match results are sent from server to PlayFab	Match stats (kills, assists, result)	Data successfully stored in backend	As expected	Pass
IT-06	Progression + Backend	XP update after match	Ensure XP calculation integrates with PlayFab storage	Calculated XP value	XP updated and reflected in player profile	As expected	Pass
IT-07	Inventory + Gameplay	Loadout retrieval before match	Verify inventory data is loaded and applied in session	PlayerID with stored loadout	Correct loadout applied in gameplay	As expected	Pass
IT-08	AI + Navigation System	AI decision integrated with pathfinding	Ensure AI decision-making correctly triggers navigation system	Enemy detection input	Bot navigates and engages correctly	As expected	Pass

IT-09	Network + Lag Compensation	Hit validation under latency	Verify lag compensation works with server validation	Simulated high latency input	Accurate hit detection despite delay	As expected	Pass
IT-10	Full System Flow	End-to-end gameplay cycle	Test complete flow from login to match completion and data saving	Player login → match → results	Full workflow executes without errors	As expected	Pass

6.2. Automated Testing

Automated testing in OmniRaid is implemented to ensure consistent, repeatable, and efficient validation of critical system components without requiring continuous manual intervention. Unlike manual testing, automated testing uses scripts and testing frameworks to execute predefined test cases, compare expected and actual results, and report outcomes automatically.

In OmniRaid, automated testing is particularly important due to the presence of real-time multiplayer interactions, server-authoritative logic, and backend data synchronization. These systems require frequent validation to ensure stability, accuracy, and performance under different conditions.

The automated testing approach in OmniRaid focuses on:

- Verifying gameplay logic using Unity Test Framework (Play Mode tests)
- Validating multiplayer synchronization using multi-client simulation
- Testing backend integration such as XP updates and inventory retrieval using PlayFab SDK

This ensures that core systems remain stable even after updates or modifications.

Table 6.2.24: Tools Used for Automated Testing

Tool Name	Tool Description	Applied on (Test Cases / Modules)	Results
Unity Test Framework	Built-in Unity testing tool for running Edit Mode and Play Mode tests using C# scripts	Gameplay mechanics, XP calculation, authentication logic	All test cases executed successfully with expected outputs
Photon Fusion Simulation (Multi-Client Testing)	Allows running multiple clients simultaneously to test networking behavior	Player synchronization, matchmaking, GameSession flow	Real-time synchronization verified across multiple clients
PlayFab SDK Testing	Used to test backend API calls for data storage and retrieval	XP update, inventory loading, player data persistence	Data correctly stored and retrieved from backend services

Table 6.2.25: Automated Test Cases

Test ID	Module Name	Test Case / Test Script	Description	Input Data	Expected Result	Actual Result	Status
AT-01	Authentication System	Automated login validation	Verify login functionality using Unity test scripts and PlayFab API	Valid PlayerID and credentials	Player successfully authenticated and session created	As expected	Pass
AT-02	Multiplayer Networking	Multi-client synchronization test	Validate real-time synchronization between multiple players	2–3 simulated clients with movement input	All clients show consistent player positions and actions	As expected	Pass
AT-03	Matchmaking System	Automated matchmaking flow	Test player queue and match creation logic	Multiple players with MMR values	Balanced match created automatically	As expected	Pass
AT-04	Gameplay System	Shooting and hit validation test	Ensure server-authoritative hit registration works correctly	Fire input events from multiple clients	Hits validated correctly and damage applied	As expected	Pass
AT-05	Progression System	XP calculation automation	Validate XP calculation logic using predefined match data	Match stats (kills, assists, time played)	Correct XP calculated and returned	As expected	Pass
AT-06	Backend Integration	XP update to PlayFab	Test automatic saving of XP after match completion	Calculated XP value	XP successfully stored and updated in player profile	As expected	Pass
AT-07	Inventory System	Inventory retrieval test	Verify items are correctly fetched from PlayFab	PlayerID	Inventory loaded correctly in game	As expected	Pass
AT-08	End-to-End Flow	Full gameplay cycle automation	Test complete flow from login to match completion	Login → Match → Gameplay → Result	Full system executes without errors	As expected	Pass

Chapter 7

CONCLUSION

7. Conclusion

The OmniRaid system has been successfully designed, developed, and implemented as a comprehensive real-time multiplayer First-Person Shooter (FPS) game utilizing a hybrid Service-Oriented Architecture (SOA). The project integrates modern game development technologies including Unity Engine [14] for the client-side, Photon Fusion for real-time networking, and PlayFab for backend services and data persistence. The primary objective of this project was to create a scalable, responsive, and competitive multiplayer environment that ensures smooth gameplay, fair interactions, and persistent player progression.

Throughout the development process, significant emphasis was placed on building a system that not only delivers an engaging gameplay experience but also maintains technical robustness and architectural efficiency. The Unity client was responsible for rendering, user interaction, and local input handling, while Photon Fusion enabled real-time synchronization through an authoritative server model. This approach ensured that all critical gameplay events, such as movement, shooting, and hit detection, were validated on the server side, thereby preventing inconsistencies and reducing the chances of cheating.

The integration of PlayFab as a backend service further strengthened the system by providing reliable cloud-based data management. Player-related data such as PlayerID (PlayFabId), inventory, loadouts, match history, XP, and levels are securely stored and retrieved, allowing seamless continuity across multiple sessions. This separation between real-time gameplay handling (Photon Fusion) and persistent data management (PlayFab) contributed significantly to the scalability and maintainability of the system.

From a gameplay perspective, OmniRaid incorporates several advanced features that enhance user experience and engagement. These include AI-driven bot behavior using pathfinding and decision-making algorithms, server-authoritative hit registration with lag compensation, and a balanced matchmaking system based on MMR and network conditions. Additionally, the progression system rewards players based on their performance, encouraging continuous participation and competitive play.

A structured implementation strategy was followed, where system design was translated into functional modules through well-defined algorithms. Core functionalities such as AI navigation, combat validation, matchmaking, and XP calculation were implemented using optimized logic to ensure performance efficiency and accuracy. The use of pseudocode and modular design further improved system clarity and maintainability.

Testing and evaluation played a crucial role in validating the reliability and performance of the OmniRaid system. Multiple testing methodologies were applied, including unit testing, functional testing, integration testing, and automated testing. Unit testing ensured that individual modules such as authentication, AI behavior, and XP calculation operated correctly in isolation. Functional testing verified that gameplay features behaved according to design requirements, while integration testing confirmed seamless interaction between Unity, Photon Fusion, and PlayFab. Automated testing, implemented using Unity Test Framework and multi-client simulations, allowed consistent validation of multiplayer synchronization and backend operations.

The results of these testing processes demonstrated that the system performs reliably under various conditions, including real-time multiplayer scenarios. Player actions are accurately synchronized

across clients, backend data is consistently stored and retrieved, and gameplay remains responsive even in the presence of network delays. These outcomes confirm that OmniRaid meets its intended functional and non-functional requirements, including performance, scalability, and user experience.

Furthermore, the project highlights the practical application of modern game development practices, particularly the use of cloud services, real-time networking frameworks, and modular system design. It demonstrates how complex multiplayer systems can be effectively developed by integrating specialized technologies, each handling a specific aspect of the system. This approach not only improves system performance but also simplifies future enhancements and maintenance.

In conclusion, OmniRaid successfully achieves its goal of delivering a robust, scalable, and engaging multiplayer FPS experience. The system is well-structured, thoroughly tested, and capable of supporting real-time interactions in a competitive gaming environment. The project serves as a strong foundation for further development and showcases the potential of combining Unity, Photon Fusion, and PlayFab in building advanced multiplayer games.

7.1. Future Work

While the current version of OmniRaid successfully implements the core features of a multiplayer FPS game, there are several opportunities for future enhancements that can further improve gameplay experience, scalability, and technical sophistication.

7.1.1. Advanced Matchmaking System

In future versions, the matchmaking system can be enhanced by incorporating advanced skill-based algorithms and machine learning techniques. Instead of relying solely on MMR and ping, additional factors such as player behavior, win/loss patterns, and playstyle can be analyzed to create more balanced and competitive matches. This would significantly improve fairness and player satisfaction.

7.1.2. Dedicated Server Infrastructure

Currently, the system utilizes Photon Fusion's networking model effectively; however, future improvements could include deployment on dedicated cloud servers for better scalability and performance. This would allow OmniRaid to support a larger number of concurrent players and reduce latency in geographically distributed environments.

7.1.3. Enhanced AI and Bot Intelligence

The AI system can be further improved by implementing advanced behavior models such as reinforcement learning or adaptive AI. Bots could learn from player behavior, adjust their strategies dynamically, and provide a more challenging and realistic gameplay experience. This would be particularly useful in training modes or low-player scenarios.

7.1.4. Competitive Ranking and Leaderboards

Future versions can introduce a global ranking system and leaderboards, allowing players to compete at a higher level. Rankings based on performance metrics such as kill/death ratio, win rate, and objectives completed can enhance competitiveness and player engagement.

7.1.5. Cross-Platform Support

OmniRaid can be extended to support cross-platform gameplay, enabling players on different devices (PC, mobile, consoles) to play together. This would significantly expand the player base and improve accessibility.

7.1.6. Advanced Analytics and Player Insights

Integrating advanced analytics tools can provide insights into player behavior, gameplay patterns, and system performance. This data can be used to optimize game balance, improve matchmaking, and enhance overall user experience.

7.1.7. In-Game Economy and Monetization

Future enhancements may include the introduction of an in-game economy system, allowing players to purchase cosmetic items, battle passes, or premium features. This would provide a sustainable monetization model for the game.

7.1.8. Enhanced Security and Anti-Cheat Mechanisms

As multiplayer games are prone to cheating, future improvements can include advanced anti-cheat systems, such as behavior analysis, server-side validation enhancements, and real-time anomaly detection. This will ensure fair gameplay and protect the integrity of the system.

7.1.9. Voice Chat and Social Features

The addition of real-time voice chat and social features such as friend lists, team formation, and in-game messaging can significantly enhance player interaction and collaboration.

7.1.10. Dynamic Maps and Game Modes

Future versions can introduce dynamic environments and multiple game modes, such as capture the flag, team deathmatch, or survival modes. This would increase replayability and keep the gameplay experience fresh and engaging.

8. References

- [1] E. Games, "Photon Fusion – High Performance Multiplayer Networking," 2024, Available: <https://doc.photonengine.com/fusion/>.
- [2] Newzoo, "Global Games Market Report," 2024.
- [3] E. Z. a. K. Salen, Rules of Play: Game Design Fundamentals, MIT Press, 2004.
- [4] M. L. a. R. Z. R. Hunicke, "MDA: A Formal Approach to Game Design", in *AAAI Workshop*, 2004..
- [5] I. M. a. J. Funge, Artificial Intelligence for Games, vol. 2nd ed, CRC Press, 2009.
- [6] Microsoft, "PlayFab Documentation," 2024. Available: <https://learn.microsoft.com/en-us/gaming/playfab/>.
- [7] P. C. a. R. K. L. Bass, Software Architecture in Practice, vol. 3rd ed, Addison-Wesley, 2012..
- [8] Sommerville, Software Engineering, vol. 10th ed. Pearson, 2015.
- [9] Norman, The Design of Everyday Things, Basic Books, 2013.
- [10] J. Schell, The Art of Game Design: A Book of Lenses, CRC Press, 2019.
- [11] P. A. a. J. Offutt, Introduction to Software Testing., Cambridge University Press, 2016.
- [12] G. M. a. D. Graham, "Game Coding Complete," vol. 4th ed, no. Cengage Learning, 2012.
- [13] .. [. Microsoft, "“Visual Studio Code Documentation,”," 2024. Available: Available: <https://code.visualstudio.com/docs>.
- [14] U. Technologies, “Unity Manual” 2024.Available: Available: <https://docs.unity3d.com/>